

BUT1-R&T-R209-WEB2

Initiation au développement Web

Présentation de la ressource R209 et SAé-23

Pré requis => WEB1 / ALGO / BDD

CM	TD	TP	SAé encadré	SAé non encadré
-	14H	14H	12H	16H

- 1 note de TP (QCM)
- 1 test de TP (2H)
- 1 note de SAé

PHP	Javascript	HTML
	 JavaScript	

SQLite	PDO
	

Ressource R209		Initiation au développement Web	
Semestre	2		
Heures	Formation encadrée	24h, dont 15h de TP	
Compétence(s) ciblées, et apprentissage(s) critique(s) couvert(s)			
RT1-Administrer Niveau 1	RT2-Connecter Niveau 1	RT3-Programmer Niveau 1	
AC0112 Comprendre l'architecture des systèmes numériques et les principes du codage de l'information AC0114 Maîtriser les rôles et les principes fondamentaux des systèmes d'exploitation afin d'interagir avec ceux-ci pour la configuration et administration des réseaux et services fournis		AC0311 Utiliser un système informatique et ses outils AC0312 Lire, exécuter, corriger et modifier un programme AC0313 Traduire un algorithme, dans un langage et pour un environnement donné AC0314 Connaître l'architecture et les technologies d'un site Web AC0315 Choisir les mécanismes de gestion de données adaptés au développement de l'outil AC0316 S'intégrer dans un environnement propice au développement et au travail collaboratif	
SAÉ concernée(s)	SAÉ23 Mettre en place une solution informatique pour l'entreprise SAÉ24 Projet intégratif		

Prérequis	R107 Fondamentaux de la programmation R109 Introduction aux technologies Web R207 Sources de données R208 Analyse et traitement de données structurées
Descriptif	Le professionnel R&T peut être amené à développer, pour ses besoins personnels ou pour ses collaborateurs, un site Web, par exemple pour fournir une interface de présentation du réseau informatique : compétence «Créer des outils et applications informatiques pour les R&T» (RT3). Il doit en appréhender tous les éléments : il doit aussi bien connaître les protocoles de communication du Web que veiller à la sécurité de ceux-ci. Il doit également pouvoir accéder, traiter et afficher des informations provenant de différentes sources de données telles que des SGBD, des API ou des fichiers structurés. La présente ressource contribue aux apprentissages critiques mentionnés précédemment.
Contenus	• Introduction au protocole HTTP. • Mise en forme de pages Web : ◦ balises HTML avancées ; ◦ structure d'une page avec son DOM ; ◦ CSS avancé ou Framework ; ◦ initiation au dynamisme côté client (JavaScript, bibliothèques comme jQuery) • Scripts côté serveur. • Eléments d'interaction client-serveur (requête HTTP, URL, formulaire). • Interrogation d'un SGBD ou d'une API. • Sensibilisation à la sécurisation de sites : failles XSS, XSS stockée, injections SQL. L'utilisation de l'anglais est préconisée dans la documentation du code.
Mots-clés	Web, Développement, Algorithmes, SGBD, API, Sécurité, Client-serveur.

1. Introduction au langage PHP

1.1. Historique de PHP

- Rasmus Lerdorf a créé PHP en 1995.
- Le but était de faciliter des tâches répétitives.
- Différentes versions se sont suivies, chacune apportant des améliorations (rapidité, gestion des bases de données,...).

“ La version actuelle est la 8.3.2 du 18/01/2023 ”

- PHP est le langage de programmation côté serveur le plus utilisé sur le web. **75% de tous les sites web reposent sur PHP**
- langage populaire parmi les programmeurs et les développeurs web en raison de son utilisation généralisée.

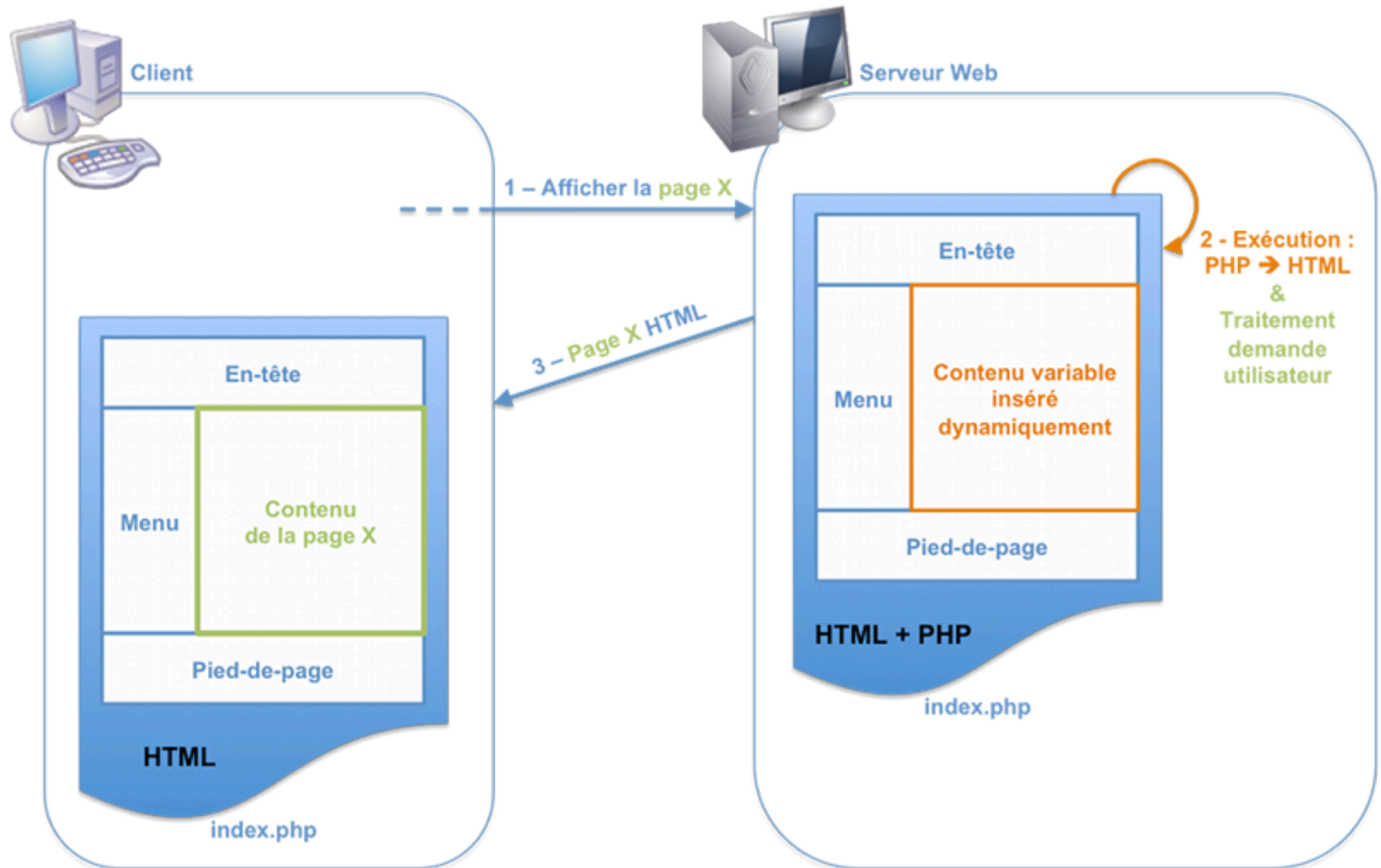
1.2. Caractéristiques de PHP

- Langage «facile à apprendre» (similitudes avec les langages C, Java, ...).
- Possibilité de programmer en objet sans être obligatoire.
- Interfaçage avec la plupart des moteurs de bases de données existants.
- API très riche (création d'image, de document PDF, ...).
- Applications PHP libres disponibles (forums, webmail, sondages, ...).
- Communauté de développeurs très dynamique.
- PHP est distribué avec Apache et MySQL (AMP) en Open source
 - Sous Linux : LAMP (Linux Apache Mysql PHP).
 - Sous Windows : WAMP (Windows Apache Mysql PHP) : xampp, easyPHP

1.3. Principe de fonctionnement

- Un fichier PHP est un fichier HTML dans lequel on ajoute des instructions PHP placées entre les balises `<?php` et `?>`. Il est sauvegardé avec l'extension .php
- Lorsque l'utilisateur (un navigateur) demande un fichier PHP au serveur, celui ci commence par vérifier s'il existe.
- Si le fichier existe, le serveur le transmet à l'interpréteur PHP. Celui ci exécute le code PHP et retourne le résultat au serveur sous la forme d'une page HTML.

“ Le serveur renvoie ensuite la page HTML générée, débarrassée des instructions PHP. ”



Requête du client Web: <http://mon-site.org/essai.php>

```
<!DOCTYPE html><!-- Code HTML -->
<html lang="fr" >
<head><meta charset="utf-8"><title>Première page PHP</title>
</head>
<body><p><!--Début du Code PHP avec <?php -->
    <?php echo "Bonjour !" ?>
<!-- Fin du Code PHP avec ?> -->
</p></body></html>
```

Réponse du serveur: <http://mon-site.org/essai.php>

```
<!DOCTYPE html><!-- essai.php (reçu par le client) -->
<html lang="fr" >
<head><meta charset="utf-8"><title>Première page PHP</title>
</head>
<body>
    <p>Bonjour !</p>
</body></html>
```

1.4. Configuration PHP

- `phpinfo()` est une fonction qui permet de connaître la version de PHP installée sur le serveur, et toutes ses options de configuration :

```
<?php phpinfo(); ?>
```

- Option intéressante: `error_reporting` qui spécifie le niveau de rapport d'erreurs de PHP (les erreurs, ou ajout des avertissements).

```
<?php error_reporting(-1); ?> // montre toutes les erreurs
```

- Les erreurs peuvent s'afficher directement dans la page ou bien dans un fichier `log`.
- Au niveau du serveur, ces options sont spécifiées dans le fichier `php.ini`.

1.5. Variables et Types

Les noms de variables sont précédés du signe **\$**

```
<?php // pas de déclaration
$maVariable = 123;
echo $maVariable; //affiche la variable
$maVariable = 123;    // entier
$maVariable = false;  // booléen
$maVariable = 3.14159; // réel
$maVariable = "bonjour"; // chaîne
?>
```

L'opérateur d'affectation est le signe **=**

Le typage est implicite, les variables ne sont pas déclarées avant l'utilisation.

“ Possibilité de définir des constantes non préfixées par un **\$** :

```
<?php define("PI", 3.14); echo PI*2;?>
```

1.6. Chaînes de caractères

Une chaîne est délimitée par des simples quotes ou par des guillemets.

```
<?php
echo "une chaîne";
echo 'une chaîne';
$nom = 'Durand';
echo "je suis $nom"; => je suis Durand
echo 'je suis $nom'; => je suis $nom
?>
```

Avec `"`, l'expression est évaluée, pas avec `'`

```
<?php // concaténation avec le .
echo 'une ' . 'chaîne';
echo 'je suis ' . $nom;
$a = 'bonjour ' . $nom;
#Important de savoir manipuler les guillemets et les simples quotes.
?>
```

1.7. Les Fonctions

1.7.1. Définition

```
<?php
function ma_fonction($nom) {
    echo "je suis $nom";
    return "je retourne $nom";
}?>
```

1.7.2. Portée et visibilité des variables

- Une variable définie dans une fonction est locale à la fonction, elle n'a pas d'existence en-dehors de cette fonction.
- En revanche, une variable définie en-dehors de toute fonction est visible dans tout le code de la page.

1.7.3. Appel & passage des paramètres

Les paramètres peuvent être passés par valeur (c'est le cas par défaut) ou par référence. Dans ce cas, on utilise l'opérateur & :

```
function miseAJour(&$liste) {...}
```

```
<?php // appel  
$retour = ma_fonction("Dupond");  
?>
```

1.7.4. Appel et valeurs par défaut:

Il est possible d'appeler une fonction en ne précisant qu'une partie de ses paramètres. Le code suivant génère un warning à l'exécution (

Warning: Missing argument 3 for miseajour() in ...)

```
<?php function maFonction($liste, $d, $f) { ... }  
maFonction($tab,1); ... // appel de la fonction ?>
```

L'avertissement disparaît si le paramètre manquant a une valeur par défaut, en définissant la fonction ainsi :

```
function miseAJour($liste, $d, $f=10) { ... }
```

```
<!doctype html>
<html lang="fr">
<head>
    <meta charset="utf-8" />
    <title>Fonctions PHP</title>
</head>
<body>
<h1>Fonctions PHP</h1>
<?php //définition
function ma_fonction($nom) {
echo "<p>Affichage dans la fonction : je suis $nom</p>";
return "je retourne $nom";
}
// appel de la fonction
$retour = ma_fonction("Dupond");
echo "<p>Affichage lors de l'appel de la fonction : $retour</p>";
?>
</body>
</html>
```

Fonctions PHP

Affichage dans la fonction : je suis Dupond

Affichage lors de l'appel de la fonction : je retourne Dupond

1.7.5. include / require

```
include      'toto.php'  
include_once 'toto.php'
```

```
require      'toto.php'  
require_once 'toto.php'
```

include

*Ok (warning), même
si le fichier n'existe
pas*

require

*erreur si le fichier
n'existe pas*

_once



*ne pas re-inclure si le fichier
l'a déjà été*

“ Afin d'avoir un code clair et bien décomposé, il faut utiliser plusieurs fichiers de code, chacun contenant des fonctions bien précises, ayant trait à un aspect précis (traitement des chaînes, entêtes HTML, etc.) 16

Ces fichiers sont ensuite intégrés et exécutés dans le code appelant, en utilisant les instructions `include` et `require` :

```
<?php
include 'entete.php';
require 'fonctions_chaine.php';
?>
```

1.8. Les tableaux

tableau indexé		tableau associatif	
<i>index</i>	<i>valeur</i>	<i>clé</i>	<i>valeur</i>
<i>notes[3] = 10,5</i>		<i>\$notes['Dupond'] = 10,5</i>	
0	12	'Durand'	12
1	11	'Martin'	11
2	13,1	'Arnaud'	13,1
3	10,5	'Dupond'	10,5

séquence
d'entiers

simple en PHP

😊

valeurs
arbitraires

Tableau indexé <=> tableau associatif dont
les indices sont des numéros ordonnés

1.8.1. Remplir un tableau indexé

```
<?php
$villes = array('Paris', 'Dijon');
$villes[] = 'Lyon'; // [] : ajouter à la fin
?>
```

1.8.2. Remplir un tableau associatif

```
<?php
$notes = array('durand'=>12, 'martin' =>11);
$notes['dupond'] = 10.5;
$notes['arnaud'] = 13.1;
?>
```

Il est possible de connaître la taille d'une table ou d'une de ses dimensions avec `count($notes)` .

1.9. Les instructions conditionnelles

```
<?php
if ($i==0) { ... }
else { ... }

if ($i==0) { ... }
elseif ($i>10) { ... }
else { ... }

?>
<?php
switch ($i)
{
    case 0: ...;      break;
    default: ...
}

$x=$val > 5?"coucou":"blabla";
$x sera égal à "coucou" ou à "blabla"
?>
```

1.10. Les instructions itératives: boucles

1.10.1. boucle for classique

```
<?php
echo '<ul>';
for($i=0; $i<10;$i++) {
    echo "<li>ligne $i</li>";
}
echo '</ul>';
?>
```

- ligne 0
- ligne 1
- ligne 2
- ligne 3
- ligne 4
- ligne 5

1.10.2. boucle foreach pour parcourir des tableaux associatifs

```
<?php
foreach( $notes as $nom => $note_etu ) {
    //pour chaque clé ($nom) du tableau ($notes), on extrait la valeur ($note_etu)
    echo "<p>M. $nom a eu $note_etu</p>";
}
?>
```

M. durand a eu 12

M. martin a eu 11

M. dupond a eu 10.5

M. arnaud a eu 13.1

“ On récupère à la fois la clé (`$nom`) et la valeur (`$note_etu`) ”

2. Interaction Client/Serveur WEB : formulaire

2.1. Introduction

Les **formulaires HTML** permettent d'introduire de l'inter-activité avec le visiteur du site.

Les données sont

- saisies à l'aide d'éléments `HTML`
- transmises au serveur par `HTTP`
- traitées via un script écrit codé en `PHP`, `ASP`, `JSP`, `Perl`, etc.

Avant d'être envoyées au serveur, les données peuvent être vérifiées côté client par le formulaire `HTML5` + des scripts Javascript.

Ce test sera aussi fait côté serveur pour la sécurité des données transmises

2.2. Éléments d'un formulaire

Les formulaires HTML5 `<form> </form>` se composent principalement de balises `<input>`.

La balise `<form>` possède un attribut :

- `action` : qui spécifie la page qui sera demandée sur le serveur
- `method` : la méthode HTTP qui sera utilisée

```
<form action="page.php" method="get">  
<form action="" method="get"> <!-- action = "" demande la même page -->
```

Chaque balise `<input>` possède un attribut:

- `type` : spécifie le format de l'entrée (`text`, `number`, `mail` ...)
- `name` : spécifie le nom de l'entrée qui sera transmis au serveur
- `id` : identifie le nom de l'entrée dans le navigateur (donc côté client)

Une balise `<input>` avec un attribut `type="submit"` correspond à un bouton `<input type="submit">` qui permet d'envoyer les données du formulaire vers le serveur.

“ Remarque : les balises présentées ici possèdent d'autres attributs... ”

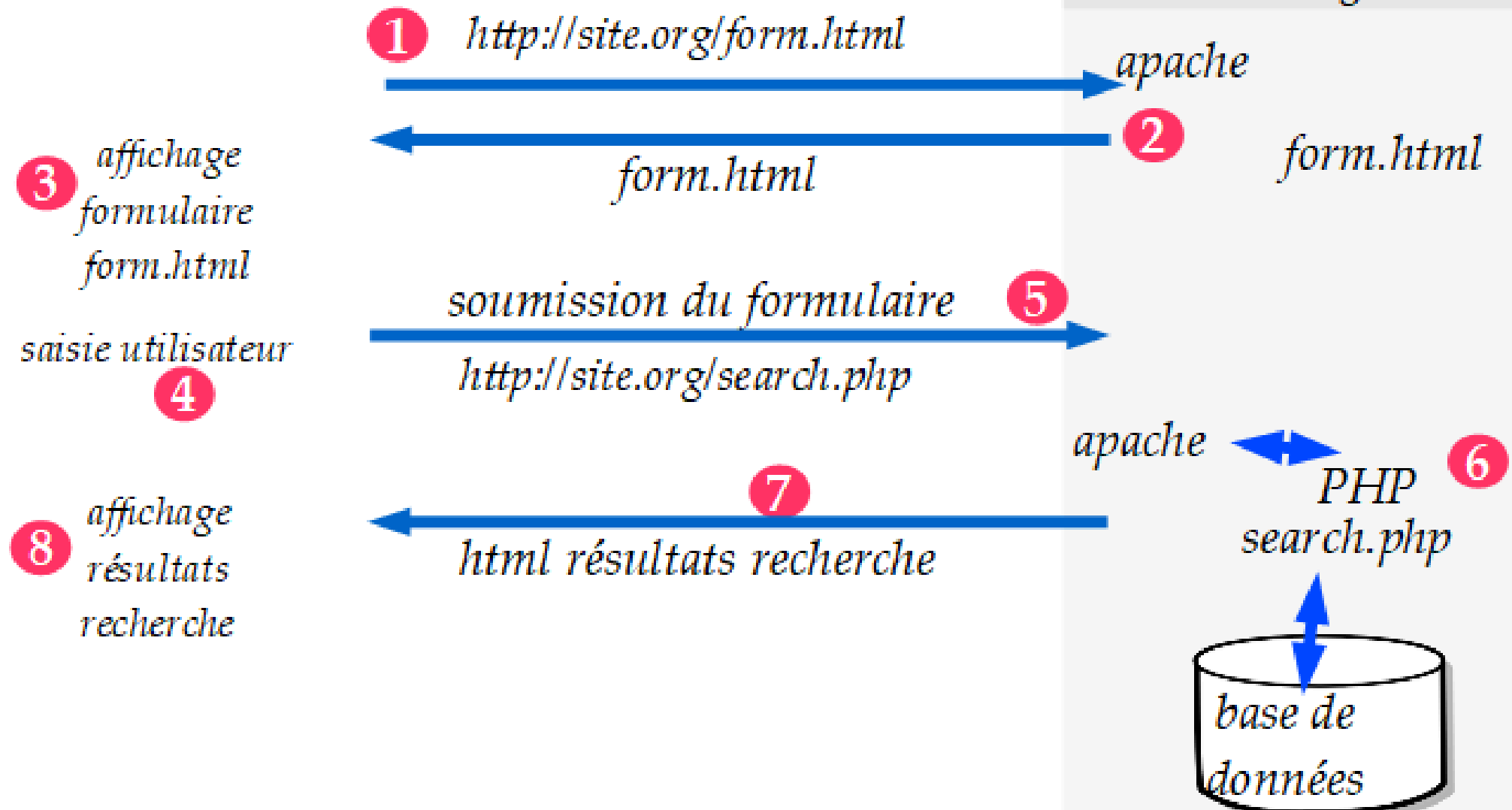
2.3. Un exemple de formulaire

```
<form method="get" action="search.php">
  <label for="in1" >termes à chercher:</label>
    <input type="text" name="ecole" id="in1" />    <br />
  <input type="checkbox" name="fr" id="fr" />
    <label for="fr">en francais?</label>    <br />
  <input type="submit" value="rechercher" />
</form>
```

```
<form method="get" action="search.php"> <label for="in1" >termes à
chercher:</label> <input type="text" name="ecole" id="in1"/>
<input type="checkbox" name="fr" id="fr"/> <label for="fr">en francais? 24
```


client (navigateur)

*serveur
site.org*



internet

client (navigateur)

RAPPEL dans le form:

```
<input type="text"  
name="ecole"  
id="id_ecole"/>
```

saisie utilisateur=
"IUT Lannion"

4



informations saisies

client serveur

recherche le mot "**IUT Lannion**"

champ texte: name="ecole"

ecole="IUT Lannion"

soumission du formulaire

5

<http://site.org/search.php>

2 méthodes
de HTTP:

GET

POST

*serveur
site.org*

soumission d'un
formulaire

apache

PHP

6

search.php récupère
ecole = IUT Lannion
dans l'entête HTTP

2.4. Les formulaires HTML5 en détail

```
<!doctype html>
<html lang="fr">
<head>
    <meta charset="utf-8" />
    <title>Formulaire HTML5</title></head>
<body>
<form action="page.php" method="get">
    <fieldset><legend>Formulaire HTML5</legend>
        <label for="idnom">Nom (Champ de texte avec placeholder)</label>
        <input type="text" name="nom" id="idnom" placeholder="Entrer votre nom" required ><br/><br/>
        <label for="idprenom">Prénom (Champ de texte avec placeholder)</label>
        <input type="text" name="prenom" id="idprenom" placeholder="Entrer votre prénom" required ><br/>
        <span>Bouton radio</span>
        <label for="idH">Homme <input type="radio" name="radioH" id="idH" value="H" checked></label>
        <label for="idF">Femme <input type="radio" name="radioH" id="idF" value="F"></label><br/>
        <span>Case à cocher</span>
        <label for="idV">Vélo<input type="checkbox" name="checkboxV" id="idV" value="checkbox" checked ></label>
        <label for="idC">Course à pied<input type="checkbox" name="checkboxC" id="idC" value="checkbox" checked ></label>
        <label for="idN">Natation<input type="checkbox" name="checkboxN" id="idN" value="checkbox" checked ></label><br/><br/>
        <label for="idnum">Age (Format numerique)</label>
        <input type="number" name="num" id="idnum" step="1" placeholder="Entrer votre age" min="0" max="120" required><br/><br/>
        <label for="iddate">Date de naissance (date)</label>
        <input type="date" name="date" id="iddate" required><br/><br/>
```

```
<label for="idemail">Champ Email</label>
<input type="email" name="email" id="idemail" required ><br/><br/>
<label for="idtel">Téléphone</label>
<input type="tel" name="tel" id="idtel" placeholder="Saisir votre tél" pattern="[0-9]*"><br/><br/>
<label for="idurl">Site WEB (Champ d'Url)</label>
<input type="url" name="siteweb" id="idurl" required><br/> <br/>
<label for="idpass">Password</label>
<input type="password" name="password" id="idpass" required><br/> <br/>
<input type="submit" name="envoyer" value="Envoyer">
</fieldset>
</form>
<footer><p>Philippe DURAND <a href="http://www.iut-lannion.fr">IUT Lannion</a></p></footer>
</body>
</html>
```

Formulaire HTML5

Nom (Champ de texte avec placeholder)

Prénom (Champ de texte avec placeholder)

Bouton radio Homme ☒ Femme ☐

Case à cocher Vélo ☒ Course à pied ☒ Natation ☒

Age (Format numerique)

Date de naissance (date)

Champ Email

Téléphone

Site WEB (Champ d'Url)

Password

Envoyer

Formulaire HTML5

Nom (Champ de texte avec placeholder)

Prénom (Champ de texte avec placeholder)

Bouton radio Homme ☒ Femme ☐

Case à cocher Vélo ☒ Course à pied ☒ Natation ☐

Age (Format numerique)

Date de naissance (date)

Champ Email

Téléphone

Site WEB (Champ d'Url)

Password

Envoyer

Veuillez compléter ce champ.

Les formulaires **HTML5** permettent un contrôle de la saisie des données par les utilisateurs.

Principalement l'attribut **required** permet d'exiger qu'un champ soit rempli avant l'envoi du formulaire.

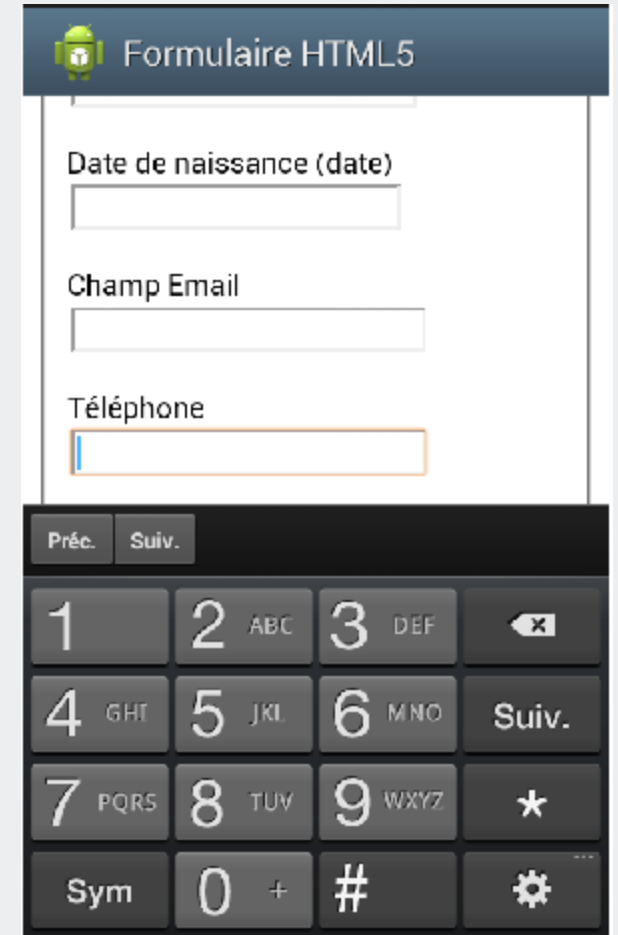
Le champ mail exige la présence d'un @, ce n'est pas suffisant pour contrôler une adresse mail et on peut affiner avec l'attribut **pattern**. Exemple d'un pattern simple pour le numéro de téléphone `pattern="[0-9]*"`.

“ Complément: On formalise le **pattern** grâce aux expressions régulières. Elles permettent de définir des contraintes sur les données attendues. Pour info http://fr.wikipedia.org/wiki/Expression_rationnelle ”

Selon les navigateurs, le rendu n'est pas le même en fonction des balises

`<input>` Illustration avec le `type="tel"`

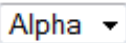
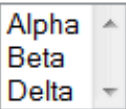
Il s'agit d'une déclinaison d'un champ de type `text`. Aussi, aucun format spécifique n'est attendu par le navigateur. Il n'y a donc pas de `pattern` précis qui vérifierait si au moins un chiffre est renseigné (il faut utiliser `pattern= "[0-9]*"` pour tester au moins la saisie de chiffres). Sur les SmartPhones, l'entrée du numéro de téléphone est facilitée par le basculement à un clavier de type numérique.



Rq : Il ne se passe rien avec Firefox sous Windows

A titre de documentation

Objet	Prévisualisation	Code HTML 5
Texte simple	Texte	<code><label>Texte</label></code>
Bouton simple	Bouton	<code><input type="button" name="nom" value="Bouton"></code>
Bouton image	bimage	<code><input type="image" src="image/bimage.jpg"></code>
Champ de texte	rien	<code><input type="text" name="texte" value="rien"></code>
Mot de passe	•••••	<code><input type="password" name="monpass" value="12345"></code>
Date	<input type="date" value="2013-09-29"/>	<code><input type="date" name="d" value="<?php echo \$today?>"></code>
Date et temps	<input type="datetime" value="1996-01-01T12:05:25-04"/>	<code><input type="datetime" name="dt" value="<?php echo \$now?>"></code>
Time	<input type="time" value="2013-06-27"/>	<code><time>2013-06-27</time></code>
Nombre	<input type="number" value="12345"/>	<code><input type="number" name="nombre" value="12345"></code>
Couleur	<input type="color"/>	<code><input type="color" name="couleur" value=""></code>
Champ de recherche	<input type="search"/>	<code><input type="search" name="" list="datalist" value=""></code>
Data	Dix	<code><data value="10">Dix</data></code>
Case à cocher	☒	<code><input type="checkbox" name="checkbox1" value="checkbox"></code>
Groupe radio	Choix 1 ☐ Choix 2 ☐	<code><label>Choix 1</code> <code><input type="radio" name="radio1" value="radio1"></code> <code></label></code> <code><label>Choix 2</code> <code><input type="radio" name="radio1" value="radio2"></code> <code></label></code>
Zone de texte	contenu	<code><textarea name="textarea">contenu</textarea></code>

Range		<pre><input type="range" min="-100" max="100" value="0" step="2" name="power" list="powers"> <datalist id="powers"> <option value="0"> <option value="-30"> <option value="30"> <option value="+50"> </datalist></pre>
Liste de données	<i>S'utilise avec input</i>	<pre><datalist id="identifieur"> <option value="1"> <option value="2"> <option value="3"> </datalist></pre>
Select		<pre><select name="select"> <option>Alpha</option> <option>Beta</option> <option>Delta</option> </select></pre>
Select liste		<pre><select name="select2" size="3"> <option>Alpha</option> <option>Beta</option> <option>Delta</option> </select></pre>
Menu	• Nouveau • Ouvrir • Sauver	<pre><menu type="context"> Nouveau Ouvrir Sauver </menu></pre>

Toolbar	Nouveau Ouvrir Sauver	<pre><menu type="toolbar"> <button type="button" onclick="fnew()">Nouveau</button> <button type="button" onclick="fopen()">Ouvrir</button> <button type="button" onclick="fsave()">Sauver</button> </menu></pre>
Combobox	0	<pre><input type="number" list="comboid"> <datalist id="comboid"> <option value="0"> <option value="-30"> <option value="30"> <option value="+50"> </datalist></pre>
Fichier	Parcourir... Aucun fichier sélectionné.	<pre><input type="file" name="file"></pre>
Image et légende	Caption	<pre><figure> <figcaption>Caption</figcaption> </figure></pre>
Fieldset	Titre ...Contenu...	<pre><fieldset> <legend>Titre </legend> <p>Contenu</p> </fieldset></pre>
Meter	<input type="range" value="12"/>	<pre><meter min=0 max=24 value=12>12 units</meter></pre>
Progress	<input type="range" value="50"/>	<pre><progress id="prog" max=100></pre>
Summary	Présentation terme définition	<pre><details> <summary> Présentation </summary> <dl> <dt>terme</dt> <dd>définition</dd> </dl> </details></pre>
Bouton d'envoi	Envoyer	<pre><input type="submit" name="envoi" value="Envoyer"></pre>

3. Méthodes HTTP

- **GET** et **POST** sont des méthodes d'accès du protocole HTTP et reprises dans la spécification HTML. Il en existe d'autres (PUT, DELETE, PATCH)
- Le choix de la méthode dépend de la façon dont les données sont envoyées, de la taille et la nature des données.

3.1. Méthode GET

- Le protocole HTTP-GET crée une chaîne de requête des paires **nom-valeur** et ajoute ensuite la chaîne de requête à l'URL du script sur le serveur qui traite la demande. Dans un formulaire, elle est spécifiée ainsi:

```
<form method="get" action="page.html"></form>
```

- Les données du formulaire seront encodées dans une **URL** composée du nom de la page ou du script à charger avec les données de formulaire empaquetées dans une chaîne.
- Les données sont séparées de l'adresse de la page avec le caractère **?** et entre elles par le code **&**.

Exemple: `http://www.site.fr/page.html?couleur=bleu&forme=rectangle`

- On doit utiliser **GET** quand la requête ne cause pas de changement dans les données sur le serveur, donc opère une simple lecture.
- Les données de formulaire doivent être uniquement des codes **ASCII**.
- La taille d'une **URL** est limitée par le serveur, souvent un peu plus de 2000 caractères, en comprenant les codes d'échappement.
- Lorsqu'on utilise le bouton retour, les requêtes **GET** sont exécutées à nouveau.

3.2. Méthode POST

- Le protocole `HTTP-POST` transmet les paires **nom-et-valeur** dans le corps du message de demande **HTTP**.
- La méthode `POST` n'a pas de taille limite et elle est spécifiée ainsi:

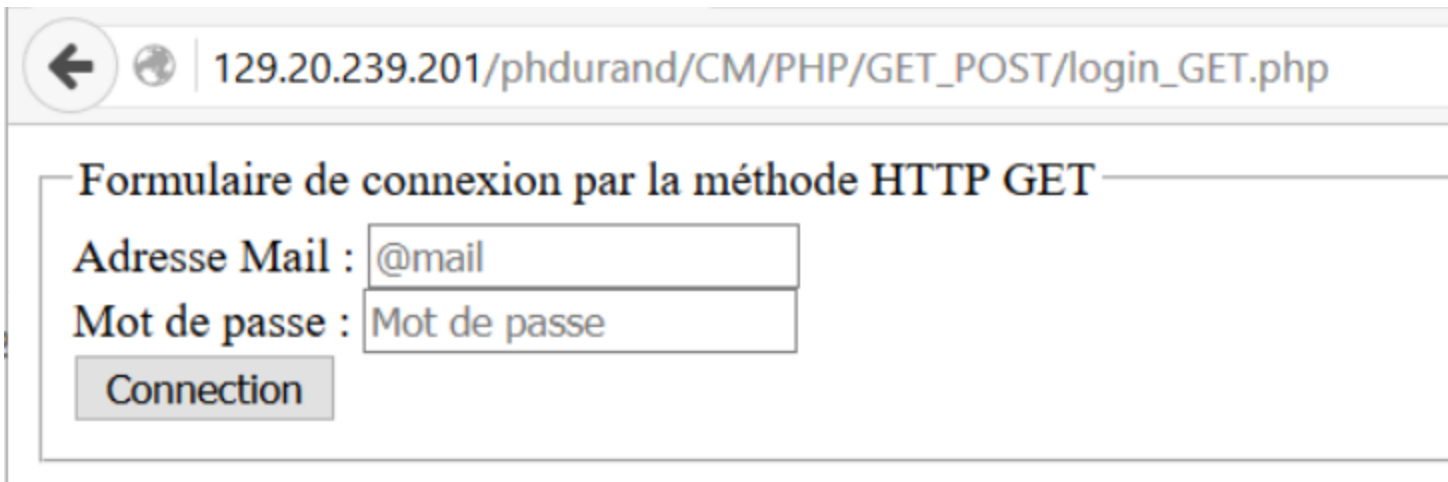
```
<form method="post" action="page.php"></form>
```

- Envoie un **en-tête** et un **corps de message** au serveur.
- Le corps est constitué des données entrées dans le champ de formulaire par l'utilisateur.
- Les données du formulaire n'apparaissent pas dans l'URL.
- **POST est indispensable pour des codes non ASCII, des données de taille importante, et elle est recommandée pour modifier les données sur le serveur.**

3.3. Exemple avec la méthode GET

On charge une page qui contient un formulaire :

```
http://129.20.239.201/phdurand/CM/PHP/GET_POST/login_GET.php
```



← 129.20.239.201/phdurand/CM/PHP/GET_POST/login_GET.php

Formulaire de connexion par la méthode HTTP GET

Adresse Mail : @mail

Mot de passe : Mot de passe

Connection

On envoie le formulaire (click sur le bouton Connection) => appelle la page (le script) indiquée dans le formulaire (balise action).

```
http://129.20.239.201/phdurand/CM/PHP/GET_POST/verif_login_GET.php?  
mail=phil%40free.fr&pass=phil&connect=Connection
```

Réponse obtenue :

```
<!DOCTYPE html>
<html lang="fr" >
<head>
<meta charset="utf-8">
<title>Identification OK</title>
</head>
<body>
<p>mail : phil@free.fr</p>
<p>pass : phil</p>
<p>connect : Connection</p>
<p>Identification OK de phil@free.fr</p></body>
</html>
```

3.4. ENTETE HTTP

L'appui sur le bouton "connexion" provoque l'envoi en HTTP du client vers le serveur avec la méthode GET

URL de la requête : http://129.20.239.201/phdurand/CM/PHP/GET_POST/verif_login_GET.php?mail=phil@free.fr&pass=phil&connect=Connection
Méthode de la requête : GET
Adresse distante : 129.20.239.201:80
Code d'état : 200 OK
Version : HTTP/1.1
Filtrer les en-têtes
En-têtes de la réponse (0,280 Ko)
Connection : "Keep-Alive"
Content-Encoding : "gzip"
Content-Length : "219"
Content-Type : "text/html; charset=UTF-8"
Date : "Thu, 10 Mar 2016 10:12:59 GMT"
Keep-Alive : "timeout=5, max=100"
Server : "Apache/2.4.10 (Debian)"
Vary : "Accept-Encoding"
X-Powered-By : "PHP/5.6.12-0+deb8u1"
En-têtes de la requête (0,442 Ko)
Host : "129.20.239.201"
User-Agent : "Mozilla/5.0 (Windows NT 10.0; WOW64; rv:44.0) Gecko/20100101 Firefox/44.0"
Accept : "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8"
Accept-Language : "en-US,en;q=0.5"
Accept-Encoding : "gzip, deflate"
Referer : "http://129.20.239.201/phdurand/CM/PHP/GET_POST/login_GET.php"
Connection : "keep-alive"

“ Méthode GET : Les paramètres sont dans l'URL

3.5. Le formulaire HTML login_GET.php

```
<!DOCTYPE html>
<html lang="fr" >
<head>
<meta charset="utf-8">
<title>Formulaire de connexion par la méthode HTTP GET</title>
</head>
<body>
<form id="form1" action="verif_login_GET.php" method="get">
<fieldset>
    <legend>Formulaire de connexion par la méthode HTTP GET</legend>
    <label for="id_mail">Adresse Mail : </label>
    <input type="email" name="mail" id="id_mail" placeholder="@mail" required /><br />
    <label for="id_pass">Mot de passe : </label>
    <input type="password" name="pass" id="id_pass" placeholder="Mot de passe" required /><br />
    <input type="submit" name="connect" value="Connection" />
</fieldset>
<!-- Ici trois balises `input` avec attribut `name="valeur"` -->
<!-- => 3 variables transmises au script `verif_login_GET.php` -->
</form>
</body>
</html>
```

3.6. La réponse formulée par le script PHP

verif_login_GET.php

```
<body>
<?php
if (!empty($_GET) && isset($_GET["mail"]) && isset($_GET["pass"])) {
    ***** Nous affichons le contenu de la variable $_GET *****
    foreach($_GET as $cle => $valeur){
        echo "<p>$cle : $valeur</p>";
    }

    if (($_GET["mail"]=="phil@free.fr") && ($_GET["pass"]=="phil"))
        echo "<p>Identification OK de " . $_GET["mail"] . "</p>";
    else { // sinon on affiche un message d'erreur
        echo "<p>L'utilisateur " . $_GET["mail"] . " n'existe pas</p>";
    }

    <a href="javascript:history.back()">Retour à la page d'authentification</a>
} // fin else
}
?>
</body>
```

- **On récupère les paramètres passés dans le tableau associatif `$_GET`,**
- Les clés de ce tableau sont `mail`, `pass` et `connect`,
- Elles correspondent aux trois attributs `name` des balises `input` du formulaire HTML

On vérifie que :

- le tableau `$_GET` n'est pas vide `!empty($_GET)`
- les clés existent `isset($_GET["mail"]) && isset($_GET["pass"])`
- On aurait pu vérifier que les données associées aux clés ne sont pas vides `$_GET["mail"] != "" && $_GET["pass"] != ""`

4. Interactions avec une Base de données en PHP

“ PHP peut s'interfacer avec de nombreux SGBD : SQLite , PostgreSQL , MySQL . ”

4.1. SQLite

SQLite n'est pas vraiment un SGBD (Système de gestion de bases de données) mais une librairie qui permet de gérer un fichier comme une base de données.

Il doit être validé dans la configuration de PHP (php.ini).

On peut utiliser une librairie PHP qui permet d'utiliser directement des fonctions d'accès à SQLite

4.2. PHP Data Objects (PDO)

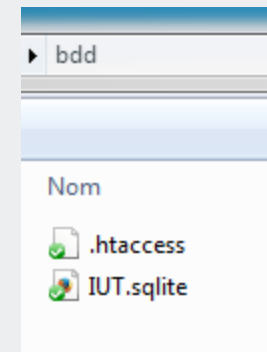
“ L'extension PHP Data Objects est une interface d'accès aux bases de données qui rend indépendant vos scripts de la base de données utilisée (sauf la connexion) parmi celles supportées (MySQL , SQLite , PostgreSQL , Oracle ...). ”

Si vous décidez de migrer vers un autre **SGBD** , il vous suffit de modifier le fichier de configuration pour que votre application s'adapte automatiquement sans avoir à toucher à votre code PHP.

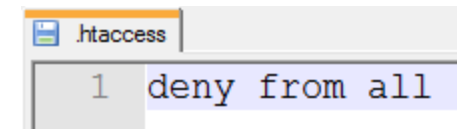
“ PDO permet d'envoyer et de récupérer le résultat de requêtes SQL de manière totalement transparente. ”

4.3. Stockage BDD SQLite

La base de données SQLite étant un fichier présent dans le répertoire de vos pages web, il faut commencer par la stocker dans répertoire qui sera inaccessible depuis un navigateur via HTTP: `http://www.site.fr/bdd/IUT.sqlite` ne doit pas retourner la bdd.



Pour cela, on la rangera dans un dossier ``bdd` qui est protégé en accès grâce à un fichier .htaccess contenant la directive `deny from all` => Erreur HTTP 403



4.4. Connexion BDD SQLite avec PHP / PDO

Pour se connecter, il suffit de créer un objet `PDO` en lui passant comme paramètre la chaîne de connexion vers la base de données

```
<?php  /*la base de données doit exister sinon elle sera créé*/  
$madb = new PDO('sqlite:bdd/cave.sqlite'); //pas de login mot de passe avec sqlite  
  
// $madb->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
  
// la connexion retourne un objet à utiliser ensuite lors des requêtes  
?>
```

4.5. Requête SELECT avec PHP / PDO : query()

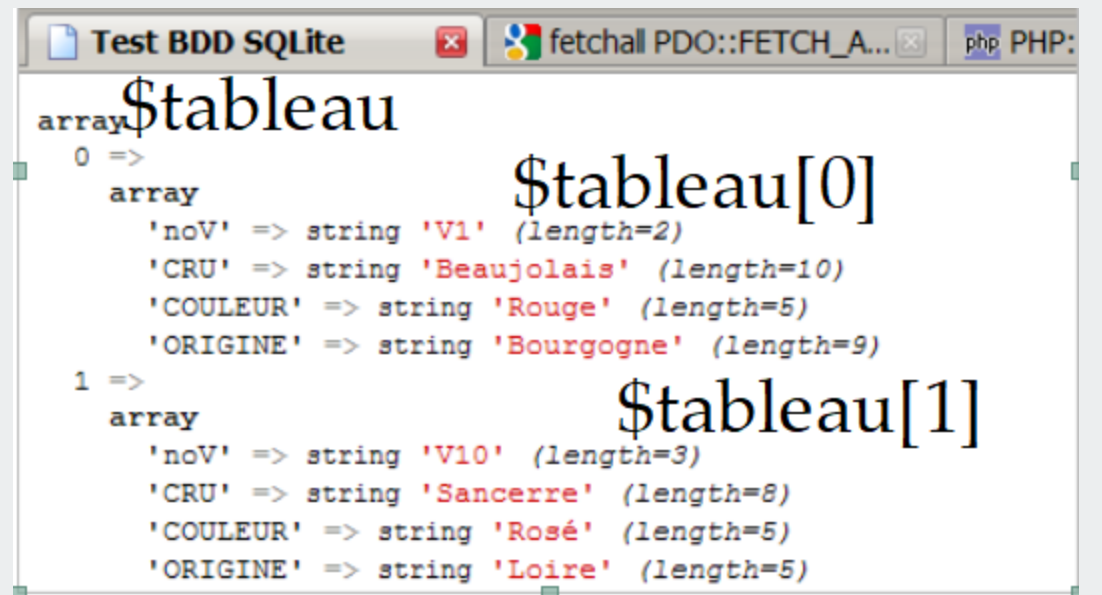
```
<?php
$madb = new PDO('sqlite:bdd/cave.sqlite');

$sql="SELECT * FROM vins";
// $sql = Chaine de caractères de la requête SQL

$res = $madb->query($sql);
// Exécute la requête SQL : retourne un objet PDO

if ($res) {
    // $res = objet PDO contenant des résultats (ou pas : requête fausse, bdd inaccessible)
    $tableau = $res->fetchAll(PDO::FETCH_ASSOC);
    // $tableau est un tableau de tableaux associatif
}
else
    echo '<p>'. 'Requête ne retournant pas de résultat ou BDD introuvable' . '</p>';
?>
```


On récupère avec la méthode (~ fonction) `fetchAll(PDO::FETCH_ASSOC)` de l'objet `$res` un tableau de tableaux associatifs (donc à 2 dimensions)



The screenshot shows a web browser window with the title "Test BDD SQLite". The address bar shows "fetchall PDO::FETCH_A..." and the PHP version "PHP: 5.6.30". The main content area displays the output of a PHP script, which is a 2D array of wine data. The array is indexed from 0 to 1. Each index contains an associative array with keys 'noV', 'CRU', 'COULEUR', and 'ORIGINE'. The values are strings representing wine characteristics. The output is formatted with syntax highlighting and variable names like \$tableau[0] and \$tableau[1] are shown next to the corresponding arrays.

```
array(2) $tableau
  0 => array(4) $tableau[0]
    'noV' => string 'V1' (length=2)
    'CRU' => string 'Beaujolais' (length=10)
    'COULEUR' => string 'Rouge' (length=5)
    'ORIGINE' => string 'Bourgogne' (length=9)
  1 => array(4) $tableau[1]
    'noV' => string 'V10' (length=3)
    'CRU' => string 'Sancerre' (length=8)
    'COULEUR' => string 'Rosé' (length=5)
    'ORIGINE' => string 'Loire' (length=5)
```

“ Il existe la méthode `fetch(PDO::FETCH_ASSOC)` lorsqu'il n'y a qu'un résultat attendu, dans ce cas on récupère directement un tableau associatif. ”

4.6. Requête INSERT avec PHP / PDO : exec()

Lorsque l'on veut modifier des éléments de la base de données, on utilise la méthode **PDO** `exec()`

```
<?php
    $clé="V11";                                $cru="Médoc";
    $couleur="Rouge";                          $origine="Bordeaux";
    $sql="INSERT INTO vins (NoV,cru,couleur,origine)
VALUES('" . $clé . "', '" . $cru . "', '" . $couleur . "', '" . $origine . "')";
    var_dump($sql);
    $retour = $madb->exec($sql);    //utilisation autre méthode PDO: exec()
    var_dump($retour);
?>
```

“ Notez qu'il faut des guillemets ici ' autour des valeurs à insérer dans la requêtes SQL ”

”

Insertion réussie => retour int 1

```
string 'INSERT INTO vins (NoV,cru,couleur,origine) VALUES ('V11','Médoc','Rouge','Bordeaux')' (length=85)
int 1
```

Insertion échouée => retour boolean false

```
string 'INSERT INTO vins (NoV,cru,couleur,origine) VALUES ('V11','Médoc','Rouge','Bordeaux')' (length=85)
boolean false
```

“ La difficulté est la bonne écriture de la requête SQL en PHP .

On prendra la bonne habitude

- d'afficher la requête en debug avec la fonction PHP `var_dump()` ,
- de la tester avant avec un client SGBD (ex : DB Browser for SQLite ,
PhpMyAdmin , ...)

”

4.7. Caractères spéciaux & requêtes SQL

Si une valeur à insérer contient un caractère spécial (ex: une guillemet ') alors la requête sera fausse.

Exemple: les variables suivantes \$clé, \$cru,\$couleur, \$origine peuvent provenir d'un formulaire.

```
<?php
    $clé="V14";$cru="L'Hospitalet";$couleur="Rouge";$origine="Languedoc";
    $sql="INSERT INTO vins (NoV,Cru,Couleur,Origine)
        VALUES ('" . $clé . "','" . $cru . "','" . $couleur . "','" . $origine . "')";
    var_dump($sql);
    $retour = $madb->exec($sql);
    var_dump($retour);
    // Le resultat est faux => la requête a échoué
?>
```

```
string 'INSERT INTO vins (NoV,cru,couleur,origine) VALUES ('V14','L'Hospitalet','Rouge','Languedoc')' (length=92)
boolean false
```

Pour résoudre le problème, il faut "échapper" les caractères spéciaux (ici la ') avec une autre ' , sinon ils sont interprétés par PHP .

On veut donc obtenir une requête SQL

```
INSERT INTO vins (NoV,cru,couleur,origine) VALUES ('V14','L'Hospitalet','Rouge','Languedoc')
```

- Pour cela, il existe une méthode PDO `$madb->quote()` qui se charge d'échapper les caractères spéciaux.
- Il est aussi possible d'utiliser des requêtes dites "préparées"

Exemple avec `quote()` pour sécuriser la requête

```
<?php $sql="INSERT INTO vins (NoV,cru,couleur,origine)
VALUES (" .
$madb->quote($clé) .",". $madb->quote($cru) .",".
$madb->quote($couleur) .",". $madb->quote($origine) .")";
//
?>
```

```
string 'INSERT INTO vins (NoV,cru,couleur,origine) VALUES ('V15','L'Hospitalet','Rouge','Languedoc')' (length=93)
int 1
```

Exemple avec les requêtes préparées

```
$sql="INSERT INTO vins (NoV,cru,couleur,origine) VALUES (?, ?, ?, ?)";
$stmt = $maBDD->prepare($sql);
$stmt->execute([$clé, $cru, $couleur, $origine]);
```

Avec des requêtes préparées, il n'est plus nécessaire d'utiliser `quote()`

4.8. Suppression

```
$sql="DELETE FROM vins WHERE noV=?"; // avec requête préparée  
$statementVin = $maBDD->prepare($sql);  
$statementVin->execute(['V12']);
```

4.9. Modification

```
$sql="UPDATE vins SET cru = ?,couleur = ?,origine = ? WHERE noV = ?";  
$statementVin = $maBDD->prepare($sql);  
$resultatVin = $statementVin->execute([$cru,$origine,$couleur,$clé]);
```

“ On remarquera qu'avec les **requêtes préparées** et l'utilisation du **?**, on passe les paramètres dans **un tableau** à la méthode **execute()** (dans le bon ordre des **?** de la requête) ”

4.10. Formulaire dynamique

Au premier chargement de la page, le formulaire s'affiche seulement

`http://localhost/bdd_select/bdd_selecthtml.php`

Sélectionner une couleur pour afficher la liste des vins

Rouge ▾

Afficher les vins

A l'envoi du formulaire, on utilise la valeur qu'il transmet pour afficher la liste (des vins) correspondante.

`http://localhost/bdd_select/bdd_selecthtml.php?couleur=Rouge`

Sélectionner une couleur pour afficher la liste des vins

Rouge ▾

Afficher les vins

Voici la liste des vins de couleur : Rouge

string(69) "SELECT CRU as Cru, ORIGINE as Origine FROM vins WHERE COULEUR='Rouge'"

Cru	Origine
Beaujolais	Bourgogne
Médoc	Bordeaux
St-Emilion	Bordeaux
Chateaufneuf du Pape	Côtes du Rhône
Médoc	Bordeaux

4.10.1. Création de la liste déroulante dynamique

Le formulaire suivant sera en partie généré par une fonction PHP qui générera les balises HTML dynamiquement.

```
<form action="" method="get"> <!-- rappelle la même page -->
  <fieldset>
    <select name="couleur">
      <option value="Rouge">Rouge</option>
      <option value="Rosé">Rosé</option>
      <option value="Blanc">Blanc</option>
    </select>
    <input type="submit" value="Afficher les vins"/>
  </fieldset>
</form>
```

```

function afficheFormCouleur() {
    $connBD=connexionBD();
    $req = "SELECT DISTINCT COULEUR FROM vins";
    $results = $connBD->query($req);
    $tab_couleurs=$results->fetchAll(PDO::FETCH_ASSOC);
?>
<h1> Sélectionner une couleur pour afficher la liste des vins</h1>
<form action="" method="get">
    <fieldset>
        <select name="couleur">
            <?php
                foreach($tab_couleurs as $tab_couleur){
                    echo '<option value="'. $tab_couleur["COULEUR"] .'">' . $tab_couleur["COULEUR"] . '</option>';
                }
            ?>
        </select>
        <input type="submit" value="Afficher les vins"/>
    </fieldset>
</form>
<?php
}

```

“ Il faut un échange (aller retour) Client/Serveur pour afficher ce formulaire dans le navigateur ”

4.10.2. Récupération du choix de ce formulaire

Ensuite l'utilisateur peut choisir un élément de la liste déroulante.

Exemple: ce script PHP récupère cette demande via la clé `couleur` transmise avec la méthode `GET`

```
afficheFormCouleur();  
// Le formulaire utilise la méthode GET  
// ex http://localhost/bdd_select/bdd_selecthtml.php?couleur=Rouge  
if(!empty($_GET) && isset($_GET["couleur"])){ // Lorsque le formulaire est appelé  
    $couleur=$_GET['couleur'];  
    echo "<h1> Voici la liste des vins de couleur : ".$couleur."</h1>";  
    $vins = listervinsParCouleur($couleur);  
//Fonction qui récupère les vins par couleur  
    if ($vins) afficheTableau($vins);  
//Fonction qui affiche dans une table HTML un tableau PHP à 2 dimensions  
    else{  
        echo "<p>Erreur de récupération des données depuis le formulaire</p>";  
    }  
}
```

4.11. PDO et les autres SGDB : exemple avec MySQL

```
<?php
//*****Connexion BD*****
function connexionBD() {
try {
    // Connection au serveur
    $dns = 'mysql:host=localhost;dbname=phdurand;port=3306'; // port n'est pas utile
    $utilisateur = 'root';
    $motDePasse = '';
    $connBD = new PDO( $dns, $utilisateur, $motDePasse );
}
catch (PDOException $e) {
    echo "Connection à MySQL impossible : ", $e->getMessage();
    die();
}
return $connBD;
}
?>
```

- Il est recommandé de créer un fichier `php` (ex `connection.php`) contenant ces lignes.
- Si l'une de vos pages nécessite un accès à la base de données, il suffira d'inclure ce fichier avec `require_once('conf/connection.php');`
- La fonction `die()` porte bien son nom, elle va tuer le script PHP (toutes les lignes situées après le `die()` ne seront jamais exécutées).

5. Cookies

5.1. Introduction

Le protocole HTTP est dit "stateless", ou autrement dit sans état.

- Pour un serveur, chaque requête qu'il reçoit est indépendante de la précédente, ainsi que de la suivante.
- Un serveur web est "bête" en quelque sorte, il écoute son port, à chaque fois qu'une requête se présente, il la traite, puis passe à la suivante.
- Ce processus est néanmoins gênant pour une application Web, car celle-ci a souvent besoin de "se souvenir" de paramètres d'un utilisateur.

“ Un cookie est un petit fichier texte (inoffensif), crée par le serveur pour être écrit sur le client (Navigateur).

À l'exception du piratage, c'est le seul cas où un site peut intervenir sur le disque dur d'un utilisateur.

1. Le client demande une page à un serveur
2. Le serveur indique au client qu'il souhaite y stocker un **cookie**, en ajoutant l'en-tête `Set-cookie` dans sa réponse au client.
3. Le client peut décider, ou non, de s'exécuter et de créer le **cookie** demandé.
4. Si le navigateur accepte, il va ensuite joindre le **cookie** à l'en-tête de toutes ses requêtes vers le même domaine.

Aux requêtes suivantes, le **client ajoute le Cookie dans l'en-tête HTTP** et le serveur recevant cet en-tête va le transmettre à **PHP** (qui récupérera ainsi les données du client).

Ainsi sur une seule requête **HTTP**, on ne peut savoir si le client accepte les cookies, ou non.

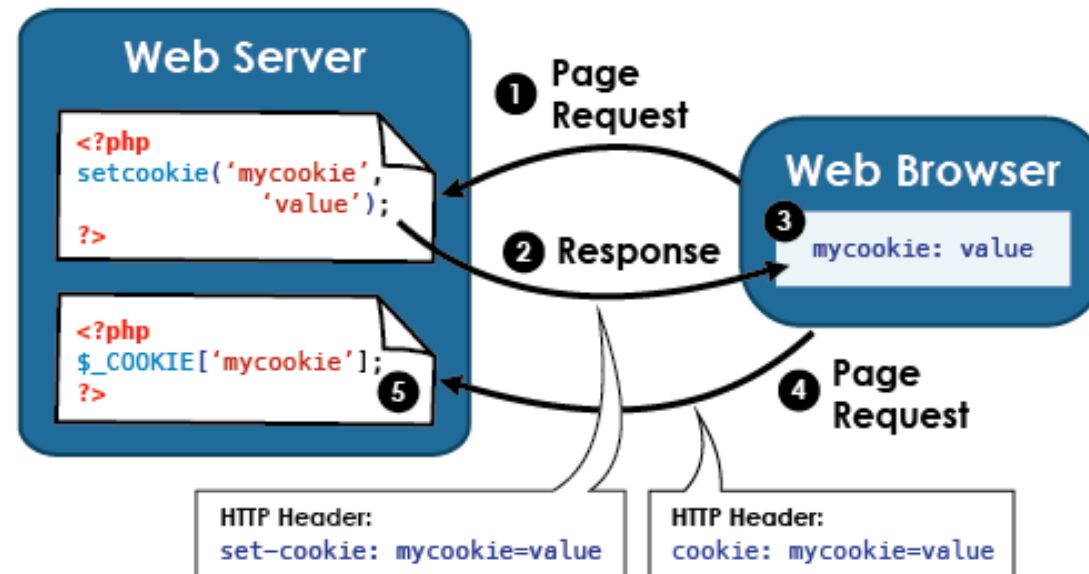
Il va être obligé de ré-adresser une requête au même serveur qui va cette fois ci regarder s'il a reçu en retour le **cookie**, ou non.

“ Un **cookie** est cependant limité en taille, 4Ko maximum; et en nombre de cookies; mais ce chiffre est généralement très grand. ”

“ On comprendra que pour stocker une quantité importante d'informations, le cookie n'est pas la solution. Il sert dans une grande majorité des cas à identifier un utilisateur. ”

5.2. Fonction `setcookie()` et variable globale `$_COOKIE`

- La fonction PHP `setcookie(name, value, expire)` permet de demander la création d'un cookie dans le navigateur
- PHP obtient les cookies reçus par le serveur WEB (caché dans l'entête HTTP).
- Lorsque PHP récupère les cookies, il les place dans un tableau global : `$_COOKIE`.



- Si une date est précisée (format `timestamp`), le navigateur client efface le cookie à l'échéance.
- Si on ne spécifie pas de date d'expiration, le **cookie** sera stocké dans la mémoire vive du client (en général), et détruit dès la fermeture du navigateur (et non de l'onglet).

“ Pratique pour retenir du client un paramètre temporaire, comme son nom, que l'on peut afficher sur nos pages. ”

5.3. Enregistrement d'un cookie dans un navigateur

Lors de l'appel de la page `cookies1_24H.php` par le client, un cookie sera enregistré dans le navigateur du client (s'il accepte).

```
<?php //Un cookie qui dure 24h
setCookie('IUT_Lannion_R&T', 'CM4 Cookies / Sessions', time()+24*3600);
?>
```

Developer Tools - http://129.20.239.201/phdurand/CM/PHP/cookies/cookies1_24H.php

Inspector Console Debugger Style Editor Performance Memory Network Storage Accessibility

Filter URLs

Status	Method	Domain	File	Cause	Type	Transferred	Size	0 ms	40.96 s
200	GET	129.20.239.201	co...	document	html	346 B	0 B	0 ms	
404	GET	129.20.239.201	favi...	img	html	cached	289 B		
200	GET	129.20.239.201	co...	document	html	346 B	0 B		

Headers Cookies Params Response Timings Stack Trace

Request URL: http://129.20.239.201/phdurand/CM/PHP/cookies/cookies1_24H.php
Request method: GET
Remote address: 129.20.239.201:80
Status code: 200 OK Edit and Resend Raw headers
Version: HTTP/1.1
Referrer Policy: no-referrer-when-downgrade

Filter headers

Response headers (346 B)

- Connection: Keep-Alive
- Content-Length: 0
- Content-Type: text/html; charset=UTF-8
- Date: Fri, 08 Mar 2019 07:49:23 GMT
- Keep-Alive: timeout=5, max=100
- Server: Apache/2.4.25 (Debian)
- Set-Cookie: IUT_Lannion_R&T=CM4+Cookies+%2...9 07:49:23 GMT; Max-Age=86400
- X-Powered-By: PHP/5.6.33-0+deb8u1

Request headers (430 B)

- Accept: text/html,application/xhtml+xml;q=0.9,image/webp,*/*;q=0.8
- Accept-Encoding: gzip, deflate
- Accept-Language: fr-FR,en-US;q=0.7,en;q=0.3
- Cache-Control: no-cache
- Connection: keep-alive
- Host: 129.20.239.201
- Pragma: no-cache
- Upgrade-Insecure-Requests: 1
- User-Agent: Mozilla/5.0 (Windows NT 10.0; ...) Gecko/20100101 Firefox/65.0

Le client appelle la page `cookies1_24H.php`

Le serveur via `Set-Cookie` dans l'entête demande au client de stocker un cookie avec **le nom, le contenu et la date d'expiration programmée.**

5.4. Exemple de test de l'existence d'un cookie

Le client appelle une autre page de l'application : `lecture_cookies.php`.

Test PHP existence **cookie** dans le tableau global associatif `$_COOKIE`

```
<?php //Lecture cookies
if (isset($_COOKIE['IUT_Lannion_R&T']))
    //la fonction PHP isset() permet de tester l'existence d'une variable.
    echo $_COOKIE['IUT_Lannion_R&T'];
else
    echo "Il n'y a pas de cookie 'IUT_Lannion_R&T' enregistré";
?>
```

if	else
CM4 Cookies / Sessions	Il n'y a pas de cookie 'IUT_Lannion_R&T' enregistré

6. Sessions

6.1. Introduction

- La **session** permet à PHP de mémoriser (coté serveur) un grand nombre de valeurs entre plusieurs requêtes d'un même utilisateur.
- Les **sessions** ne transitent pas sans arrêt entre le navigateur et le serveur (à la différence des cookies).
- Une **session** permet comme le cookie de stocker temporairement des valeurs relatives à un même internaute. Une session $\Leftrightarrow$ un fichier.
- Les **variables** de **sessions** autorisées sont les variables chaînes, entières, flottantes, caractères, tableaux, objets.
- La **durée** d'une **session** est définie par l'hébergeur.

6.1.1. Utilités des Sessions

- **Pister** le visiteur et établir des statistiques précises.
- **Transporter** aisément des variables de page en page (variable globale).
- Les sessions permettent de **conserver** des variables tant que l'internaute reste connecté au site et environ 30 minutes après (en cas de déconnexion du visiteur par exemple mais ce temps dépend de votre hébergeur : renseignez-vous grâce à la fonction `phpinfo()`).

“ En complément des cookies dans les scripts d'identification (le client n'accepte pas toujours les cookies), ”

“ la session remplace parfaitement le cookie et stocke son identifiant et son mot de passe crypté pour que le membre n'ait pas à le répéter sur chaque page. ”

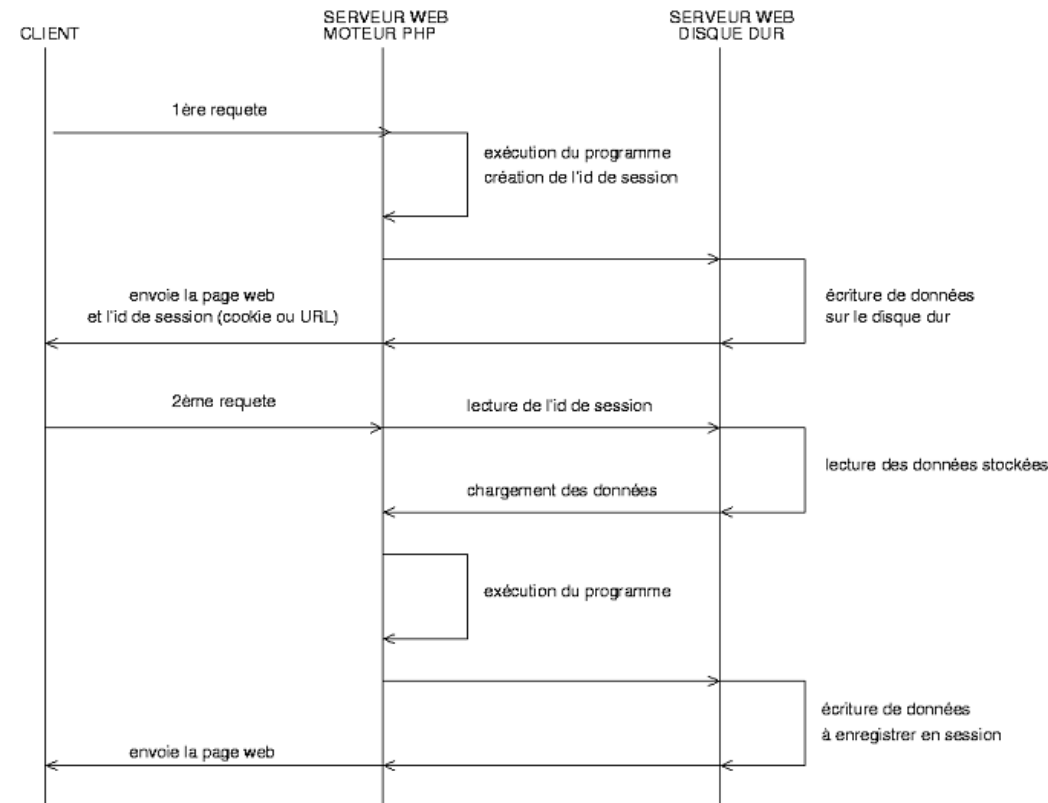
6.1.2. Unicité des Sessions

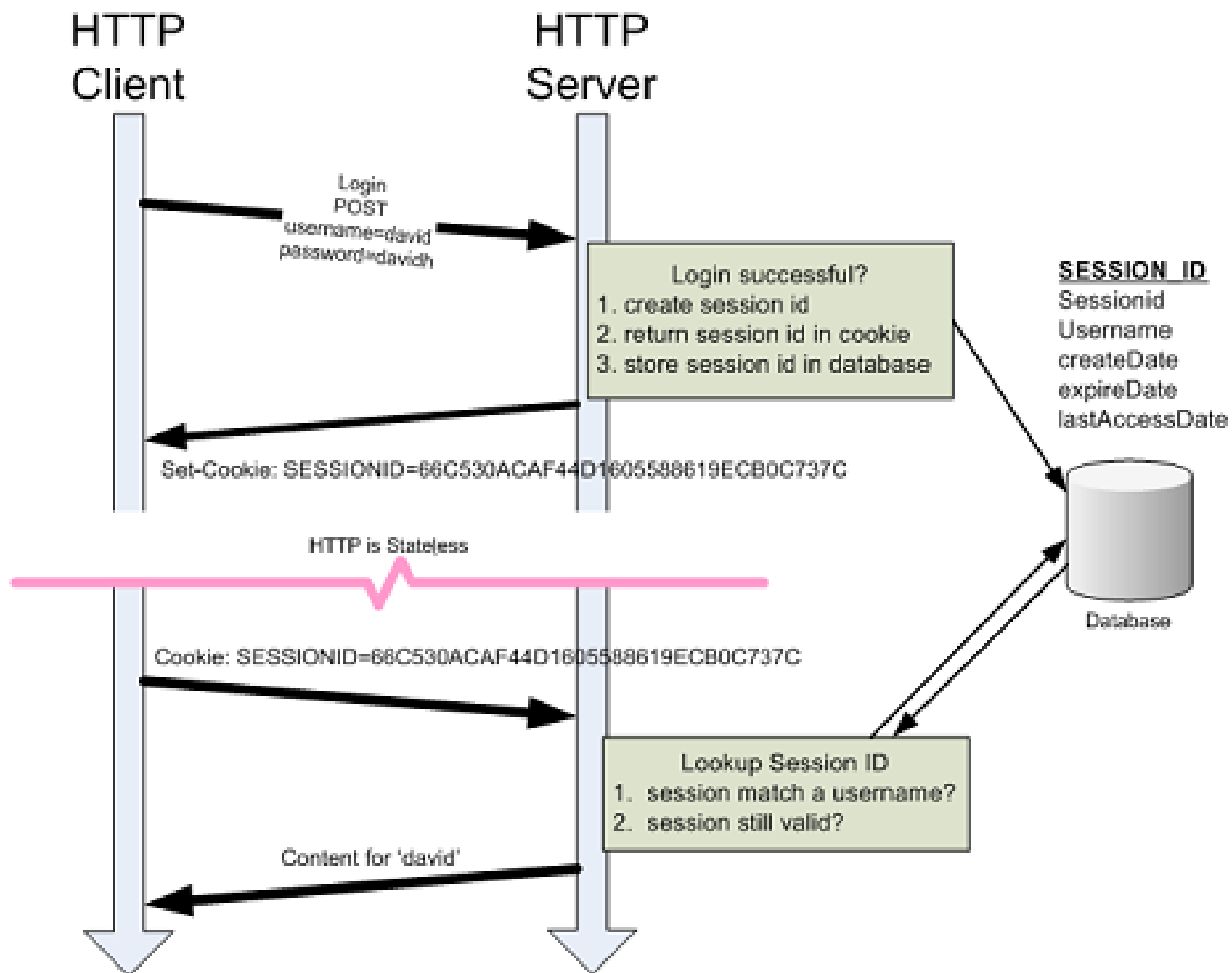
- La **session** est unique, un identifiant est généré aléatoirement pour chaque utilisateur. Cet identifiant représente l'état actuel de la session de travail.
- C'est sur l'**identifiant de session** que repose toute la sécurité du **compte de l'utilisateur**, de manière générale. Il ne s'agit que d'un petit morceau de texte, unique à chaque visiteur, généré aléatoirement et long, en général, de 32 caractères.
- Des paramètres de `php.ini` permettent de personnaliser la génération de cet identifiant.
- Pour reconnaître un internaute, dans un **protocole HTTP sans état**, son **identifiant de session doit transiter** entre chaque requête que celui-ci génère.

L'identifiant de session :

- soit il est stocké dans un cookie qui est passé de requêtes en requêtes,
- soit il transite dans l'URL avec `url?PHPSESSID=une-chaine-aleatoire`.

(Ce mode de transfert fait apparaître cette chaîne, il ne sera utilisé que lorsque le client refuse les cookies, ce qui de nos jours est très rare.)





6.2. Exemple utilisation des sessions

6.2.1. Page qui crée une session : ex1_sessions_p1.php

```
<?php session_start();  
// indispensable pour enregistrer une session sur cette page ?>  
<!DOCTYPE html> <html lang="fr" >  
<head> ... </head>  
<body> <h1>WEB2 CM Sessions : Ouverture</h1>  
<?php  
$_SESSION["login"]="prof";  
// On enregistre une variable de session "login" dans le tableau global associatif $_SESSION  
if (!empty($_SESSION))  
    echo "<p>Une session est ouverte pour l'utilisateur " . $_SESSION["login"] . " </p>";  
else    echo "<p>Pas de session ouverte</p>"  
?> ...
```

localhost/WEB2/TP/3-TD3_Sessions_Securite/4-Sessions/ex1_sessions_p1.php

WEB2 CM Sessions : Ouverture

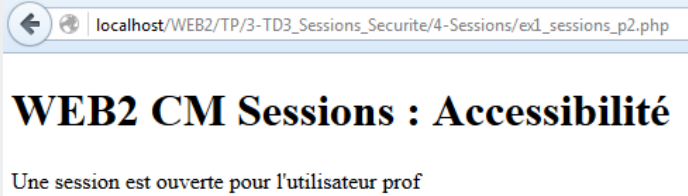
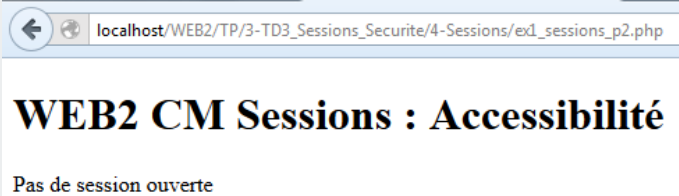
Une session est ouverte pour l'utilisateur prof

Le test est toujours vrai `$_SESSION["login"]` n'est pas vide!

6.2.2. Page accessible si une session existe : ex1_sessions_p2.php

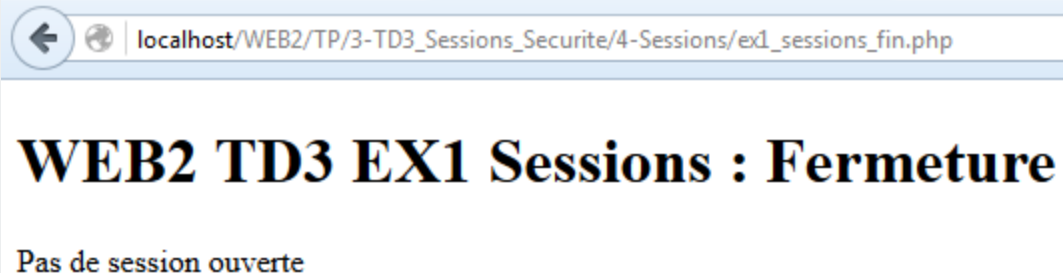
```
<?php session_start();  
// indispensable pour manipuler une session sur cette page ?>  
<!DOCTYPE html> <html lang="fr" >  
<head> ... </head>  
<body>         <h1>WEB2 CM Sessions : Accessibilité</h1>  
<?php  
if (!empty($_SESSION)) // Le test ne sera pas forcément vrai cela dépendra  
  
// si la page d'ouverture de session a été ouverte préalablement  
    echo      "<p>Une session est ouverte pour l'utilisateur " . $_SESSION["login"] . " </p>";  
else        echo "<p>Pas de session ouverte</p>" ;  
?>
```

Si la session a été créée Si la session n'a pas été créée

	
--	---

6.2.3. Page qui détruit une session : ex1_sessions_fin.php

```
<?php session_start()// indispensable pour manipuler une session sur cette page; ?>
<!DOCTYPE html>
<html lang="fr" >
<head>
<meta charset="utf-8">
<title>WEB2 CM Sessions : Fermeture</title>
</head>
<body>
<h1>WEB2 CM1 Sessions : Fermeture</h1>
<?php
$_SESSION=array();// Par sécurité, on écrase le tableau
session_destroy();// fonction PHP pour détruire une session
if (!empty($_SESSION)) echo "<p>Une session est ouverte pour l'utilisateur " .$_SESSION["login"] . " </p>";
    else                echo "<p>Pas de session ouverte</p>"
?>
```



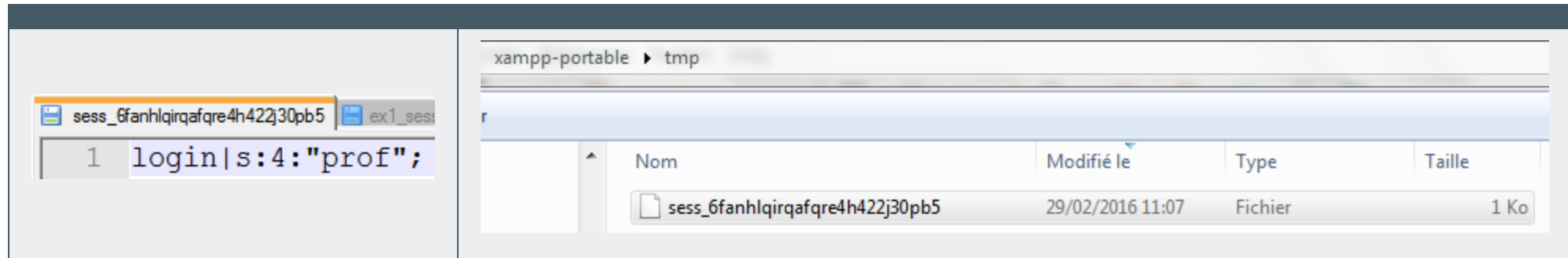
//Le test sera toujours faux car on a détruit la session juste au dessus

“ Si vous voulez détruire complètement la session, effacez également le cookie de session. Cela détruira la session et pas seulement les données de session ! ”

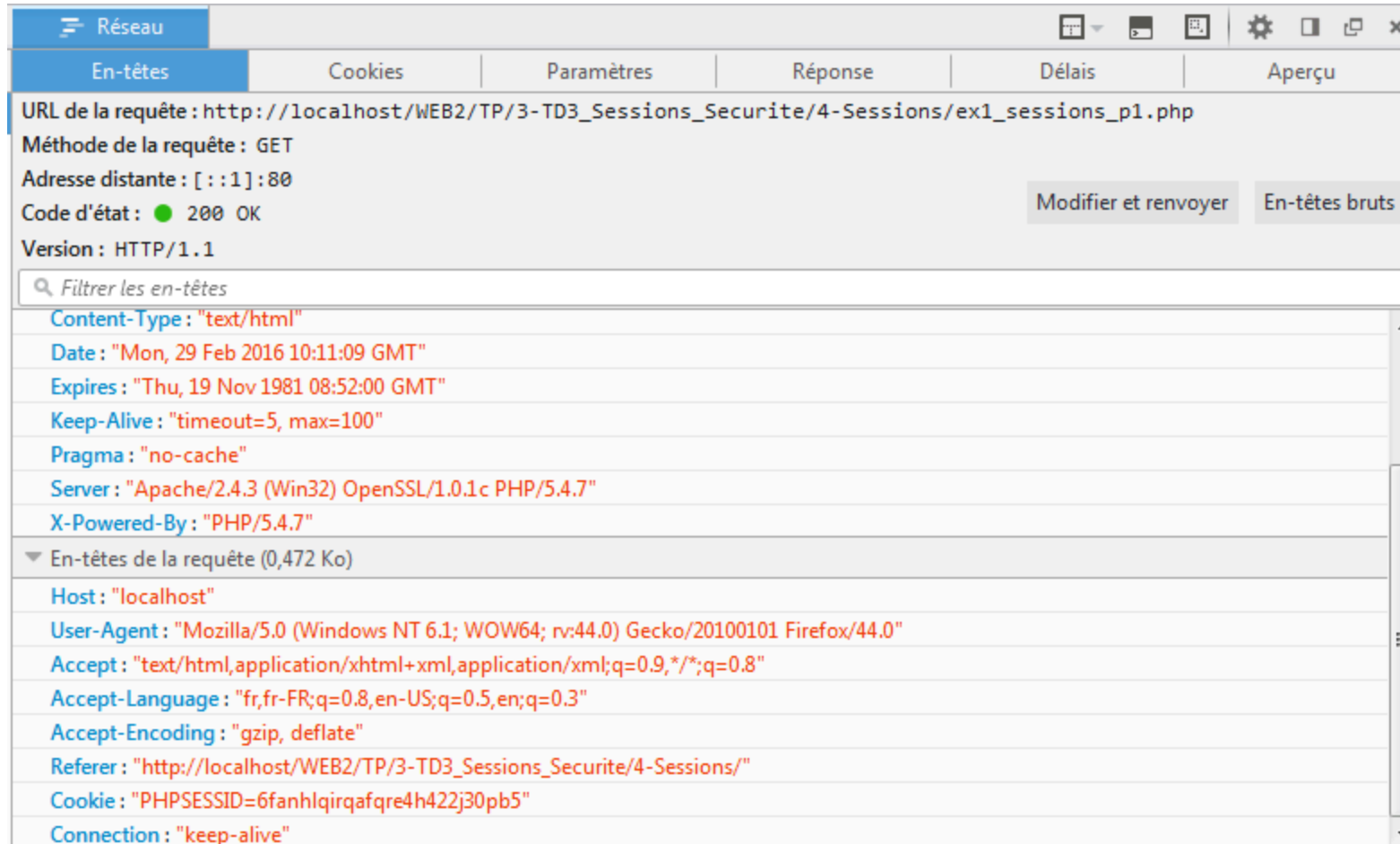
```
if (ini_get("session.use_cookies")) {  
    $params = session_get_cookie_params();  
    setcookie(  
        session_name(), '',  
        time() - 42000,  
        $params["path"],  
        $params["domain"],  
        $params["secure"],  
        $params["httponly"]  
    );  
}
```

6.2.4. Bilan de l'utilisation des Sessions

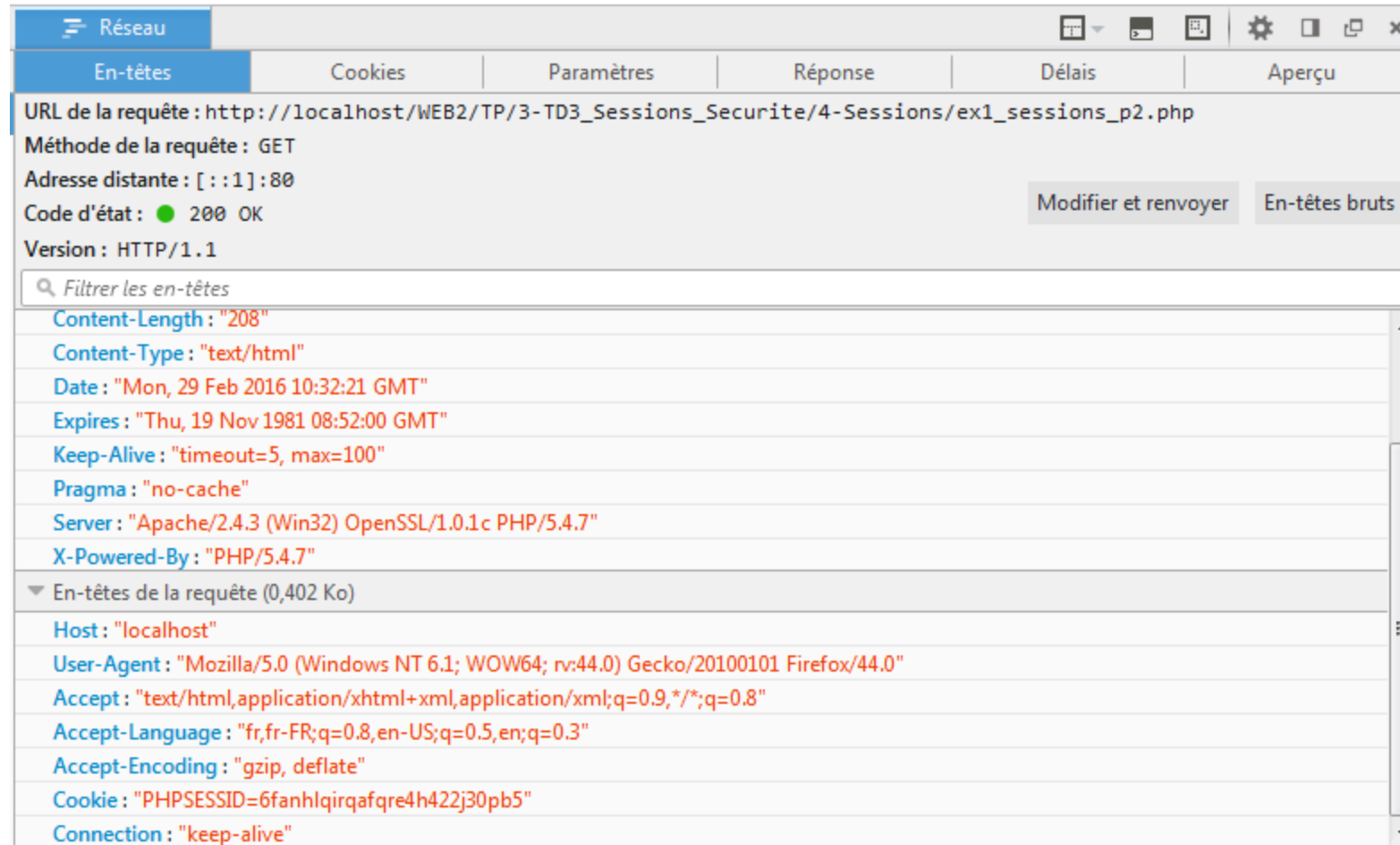
“ La session est enregistrée dans un fichier sur le serveur. ”



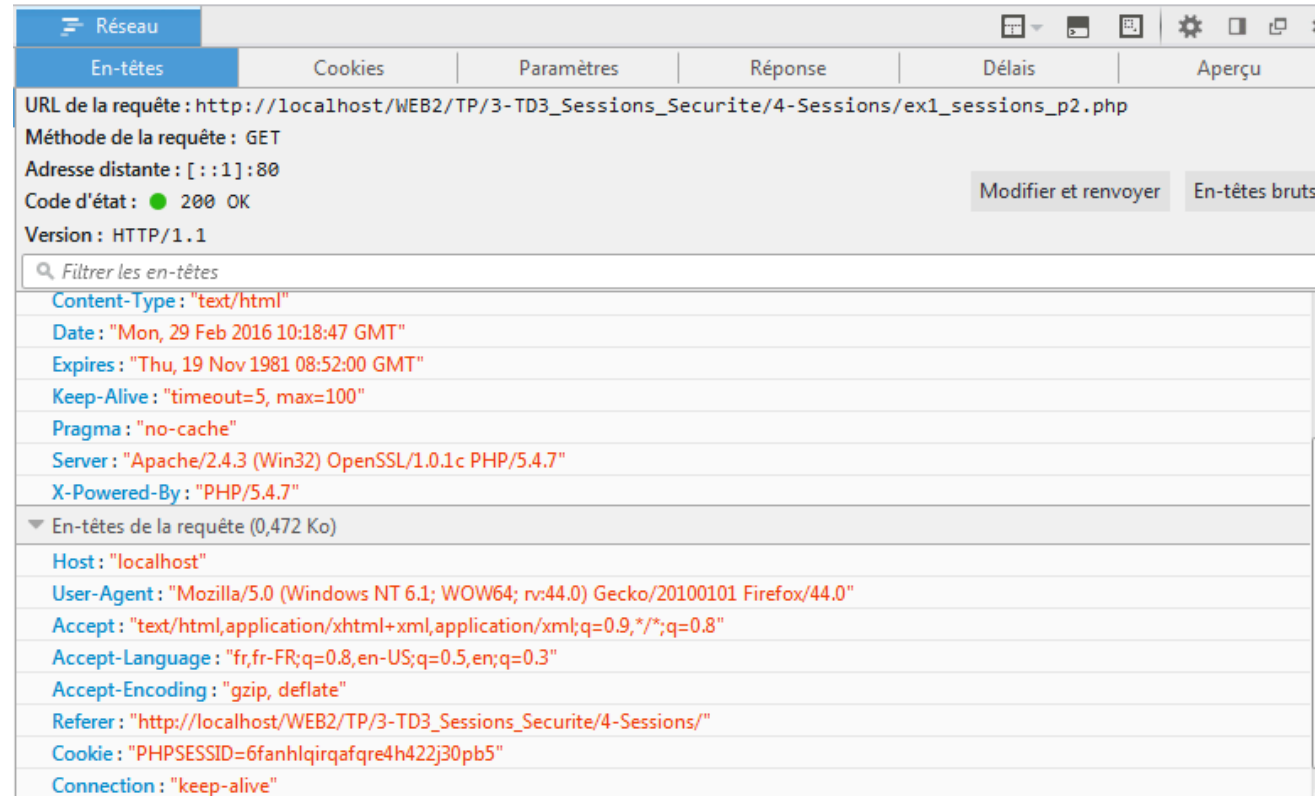
1. Appel de la page d'ouverture de session, on voit apparaître le cookie avec la clé de session **PHPSESSID**



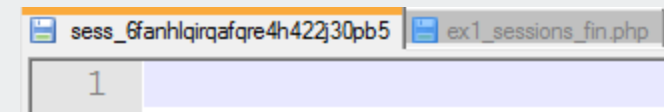
2. Appel de la page accessibilité de session, on voit apparaître le **cookie** avec la clé de session **PHPSESSID** s'il y a eu une connexion préalable.



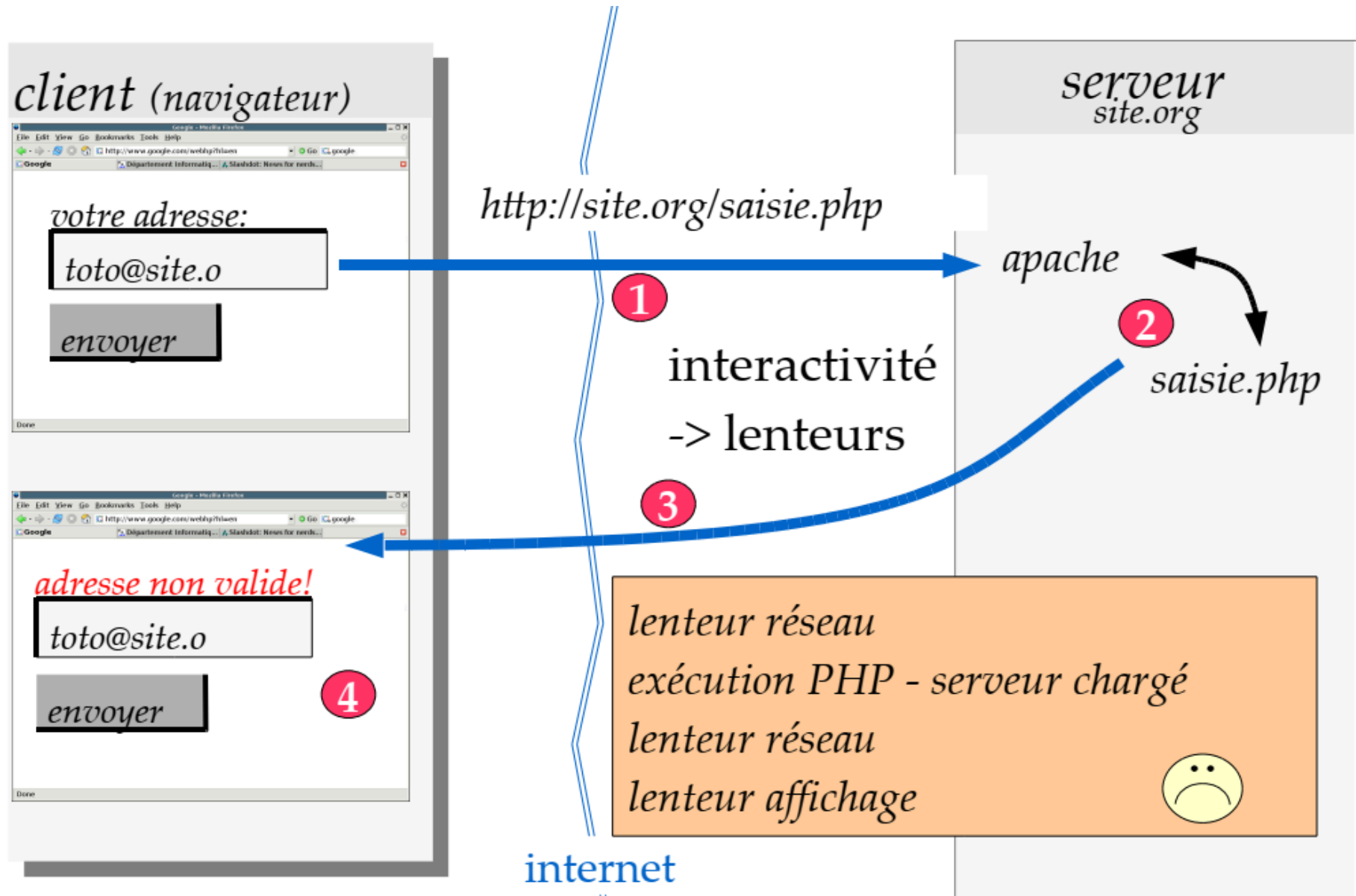
3. Appel de la même page après déconnexion => le cookie avec la clé de session PHPSESSID (la durée est fixée par l'hébergeur en général 30 min).



Le fichier de session est bien vide



7. Programmation dynamique coté client



client (navigateur)

votre adresse:

toto@site.o

envoyer

script client

adresse non valide!

toto@site.o

envoyer

internet

*serveur
site.org*

pas d'interaction
client / serveur lente



8. Le langage JavaScript

8.1. Introduction à Javascript

JavaScript est un **langage objet** de **scripts interprété** par les navigateurs.

JavaScript est un code de programmation qui **peut être inséré dans des pages HTML** exécuté par tous les navigateurs modernes y compris sur les tablettes, smartphones...

Quelques restrictions **JavaScript** :

- Pas de fonctionnalité graphique
- Impossible de lire ou écrire un fichier sur le disque de la machine où est installé le navigateur (sécurité).

“ Les scripts **Javascript** sont forcément exécutés du côté client
Les navigateurs peuvent être configurés pour rejeter ce langage. ”

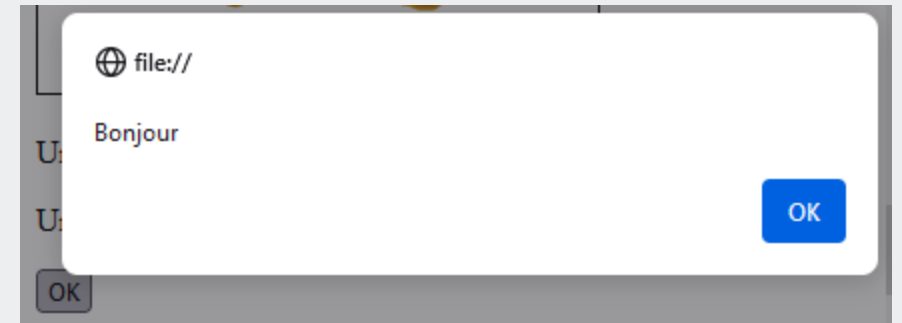
Une balise permet d'afficher du texte lorsque Javascript n'est pas autorisé par le navigateur :

```
<noscript> <p>Pour utiliser ce site, Javascript doit être activé dans votre navigateur !</p> </noscript>
```

- <https://www.w3schools.com/js/default.asp>
- <https://www.pierre-giraud.com/javascript-apprendre-coder-cours/introduction/>
- <https://developer.mozilla.org/fr/docs/Web/JavaScript>

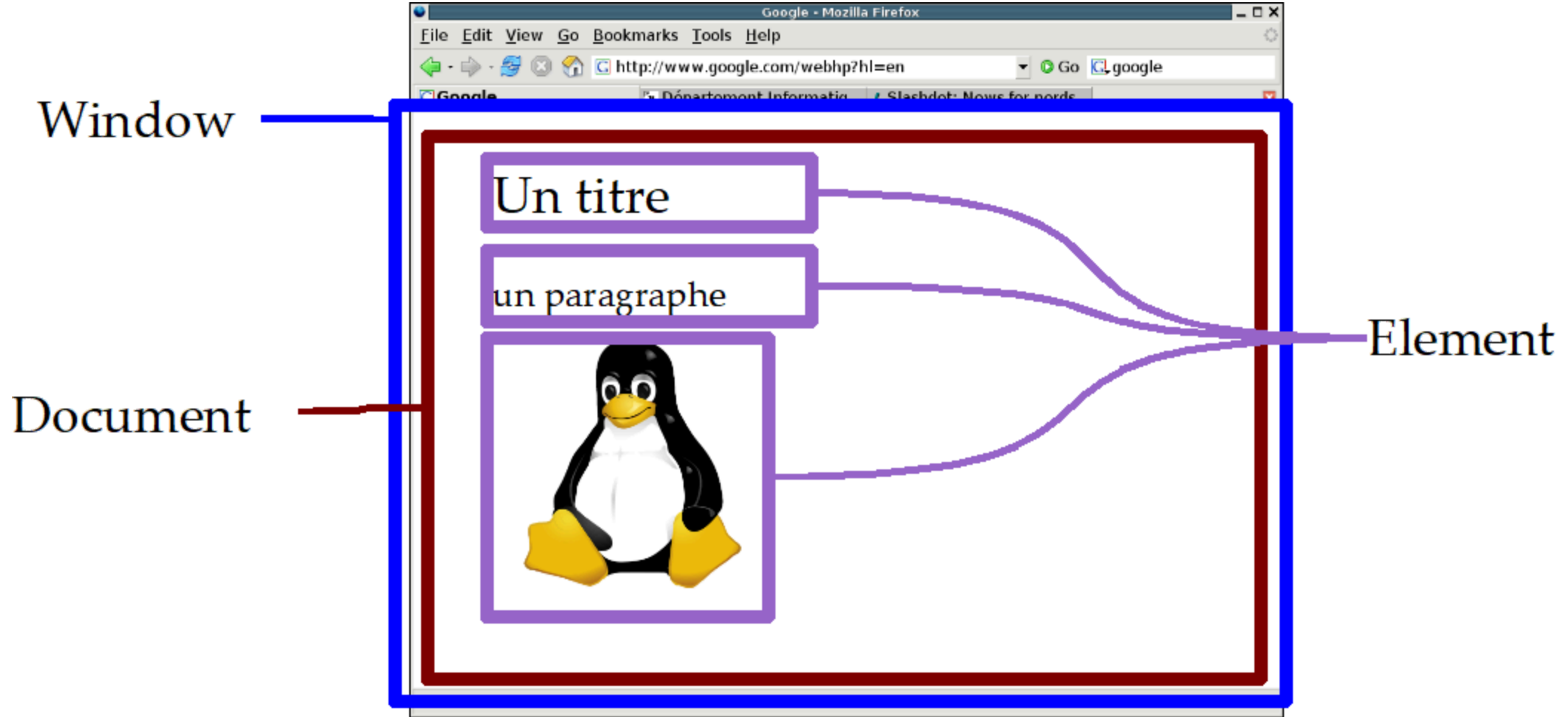
8.1.1. JavaScript: Réaction aux événements

```
<button type="button"  
onclick="alert( ' Bonjour ' )>OK</button>
```



“ **Langage événementiel => association d'actions** à des événements (clic, survol de la souris, chargement de la page, ...) ”

8.1.2. Javascript permet l'accès aux éléments HTML



- Une page HTML chargée dans un navigateur est modélisée comme un arbre.

- Le `DOM` (Document Object Model) est la norme officielle du `W3C` pour accéder à l'arbre des éléments `HTML` d'une page.
- L'utilisation de JavaScript pour manipuler le contenu des éléments `HTML` est une fonctionnalité très puissante.
- Un élément peut se choisir par son attribut `id` ou `class` ou ... et il a des propriétés.

“ `document.getElementById("un Identifiant")` est une méthode définie par `DOM` qui retourne un objet Javascript correspondant à la balise `HTML` ciblée par son `id` ”

Coté HTML

```
<p id="demo">Au chargement de la page</p>
```

Coté JavaScript entre `<script>` `</script>`

```
var ceParagraphe=document.getElementById("demo")
ceParagraphe.innerHTML="lorsque le code javascript est exécuté";
```

```
\\ ceParagraphe est un élément (objet)
\\ inner.HTML est une propriété
```

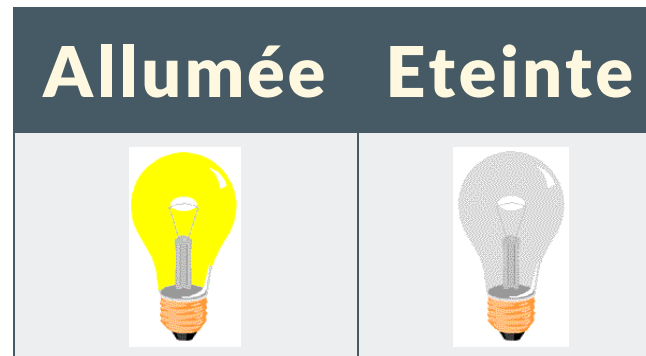
`document.getElementById("un Identifiant")` est une méthode définie par `DOM` qui retourne un objet `JS` correspondant à la balise `HTML` ciblée par son `id`

8.1.3. Un exemple JavaScript: modification d'images HTML

Modification de la source (`src`) attribut d'un élément HTML `<image>` :

```
<body>
<script>
function changeImage() {
element=document.getElementById( 'monimage' )
if (element.src.match("lampe_on.gif"))    element.src="lampe_off.gif";
else    element.src="lampe_on.gif";  }
</script>

<p>Click sur l'ampoule pour l'allumer ou l'éteindre</p> ...
```



8.2. Intégration Javascript dans une page HTML

8.2.1. Méthode 1: fichier javascript séparé (extension (.js))

```
<head>
  <script src="essai.js"      type="text/javascript">
  </script>
</head>
```

8.2.2. Méthode 2: javascript inclus avec la balise `<script>`

Dans l'entête de la page `HTML` ou dans le corps de la page `HTML`

```
<script type="text/javascript">
alert("début");// cette ligne est exécutée lors du parcours du document
function ma_fonction(chaine){
  // cette fonction est exécutée si on l'appelle !!!!!
  alert(chaine);
}
</script>
```

8.3. Exécution d'un code Javascript

Si le script est inséré dans le document `HTML`, il peut l'être à différents endroits :

- **Dans l'entête du document**

Dans ce cas le code javascript est exécuté avant l'affichage de la page.

- **Dans le corps du document**

Dans ce cas le code est exécuté pendant l'affichage de la page.

- **En réponse à des événements**

Exemples:

```
<h1 onclick="alert('bonjour!')">un titre</h1>
```

```
<body onload="ma_fonction('déclenchement onload')">
```

8.4. Ecrire dans un document HTML

```
<!DOCTYPE html>
<html>
<body>
<h1>Titre</h1>
<script>
document.write("<p>Mon premier JavaScript</p>");
</script>
</body>
</html>
```

- Utilisez `document.write()` uniquement pour écrire directement dans la production de document.
- Si vous exécutez `document.write()` après que le document a terminé le chargement, la page HTML sera écrasée.

8.5. Modifier du contenu HTML

La meilleure façon de modifier le contenu d'un élément HTML est d'utiliser la propriété **innerHTML** en utilisant la syntaxe suivante:

```
document.getElementById(id).innerHTML=nouvelleValeur;
```

8.6. Description rapide du langage JavaScript

- Script => séquence d'instructions à exécuter par le navigateur.
- Point-virgule sépare les instructions JavaScript.
- Syntaxe très proche du langage Java mais ce n'est pas le même langage.
- Langage sensible à la casse

8.6.1. Les commentaires

Comme en C : sur une ligne (`//`) ou plusieurs lignes (`/*-* .. */`).

8.6.2. Les variables mot clé var

JavaScript a des types dynamiques. Cela signifie que la même variable peut être utilisée en tant que types différents:

```
var x           // x n'est pas défini
var x = 5;      // Maintenant x est un nombre
var x = "Phil"; // Maintenant x est une chaîne
```

8.6.3. Les opérateurs:

affectation: <code>=</code>	arithmétiques : <code>==, !=, <, <=, >, >=</code>
raccourcis : <code>+=, /=, ++, -</code>	logiques : <code>&&, , !</code>

8.6.4. Les instructions conditionnelles

```
if (i==0)
{   ... } else {   ... }
switch (i) {
    case 0: ...;
        break;
    default: ...
}
```

8.6.5. Les instructions itératives

for (i=0; i<10; i++) { ... }	while (a<b) { ... }
do { ... } while (a < b);	for (i in age) { ch=i+" : "+age[i]; }

8.6.6. Les fonctions

- Les fonctions sont définies à l'aide du mot-clé `function`.
- Une fonction peut renvoyer une valeur en utilisant `return`
- Les paramètres sont séparés par des virgules, sans préciser leur type :

```
function survoleLien(lien) {  
    ...  
}
```

L'appel de la fonction se fait obligatoirement en utilisant des parenthèses :

- `parcours(tab, 10, 20);`
- `recompte();`

8.6.7. les erreurs en JavaScript : `try and catch`

Si une erreur se produit, le moteur `JavaScript` va normalement s'arrêter et générer un message d'erreur.

On peut écrire dans la console **JS** avec `console.log("message")` ; pour voir les erreurs.

- La clause `try` vous permet de définir un bloc de code à tester qui peut engendrer des erreurs en cours d'exécution.
- La clause `catch` vous permet de définir un bloc de code à exécuter si une erreur se produit dans le bloc `try`.

Ces instructions sont toujours couplées.

```
try { //code à contrôler
}
catch(err) { //Gestion des erreurs
}
```

8.6.8. Javascript est un langage de programmation orientée objet

“ **"Tout" est un objet** en JavaScript : une chaîne, un nombre, un tableau, une date

Un objet est une donnée, avec des propriétés (valeurs) et des méthodes (fonctions). ”

Avec cette variable Javascript `var texte = "Bonjour";`, vous créez un objet JavaScript de la classe String qui possède une propriété `length`.

L'objet String a également plusieurs méthodes.

- Propriétés: ici `texte.length` est égale à 7
- Méthodes: `texte.indexOf()`, `texte.replace()`, `texte.search()`

8.6.9. Les tableaux

Les tableaux sont des objets qu'il est possible de déclarer de différentes façons :

```
var mois = new Array(12);  
mois[0] = "Janvier";  
  
var coords = [10, 12, 15];  
  
var jm = new Array([10, 12], [15, 13, 25]);  
var jm2 = [[10, 12], [15, 13, 25]];  
  
var age = new Array();  
  
age["Pierre"]=15; //peut être associatif
```

8.7. Recherche d'éléments HTML

En Javascript on a vu que l'on peut,

Trouver les éléments HTML par id

```
var x=document.getElementById("intro");
```

Cet exemple recherche l'élément avec `id = "intro"`

On peut aussi

Trouver des éléments par nom de balise HTML

```
var x=document.getElementById("main");  
var y=x.getElementsByTagName("p");
```

Cet exemple recherche l'élément avec `id = "main"`, puis retrouve à l'intérieur de cet élément d'`id "main"` tous les éléments de type `<p>`.

`y` est un tableau contenant tous les objets ciblés.

Trouver les éléments HTML par nom de classe

```
var z = document.getElementById('main').getElementsByClassName('test');
```

Cet exemple recherche l'élément avec `id = "main"`, puis retrouve tous les éléments à l'intérieur dont l'attribut `class="test"`.

`z` est un tableau contenant tous les objets ciblés.

8.8. Les événements Javascript

8.8.1. Les événements JavaScript permettent l'interaction

Action	Gestionnaire	Action	Gestionnaire
bouton souris	<code>onclick</code>	bouton souris (double)	<code>ondblclick</code>
mouvement souris	<code>onmousemove</code>	touche clavier	<code>onkeypress</code>
chargement page	<code>onload</code>	survol de souris	<code>onmouseover</code> et <code>onmouseout</code>
modification de la valeur	<code>onchange</code>	soumission du formulaire	<code>onsubmit</code>

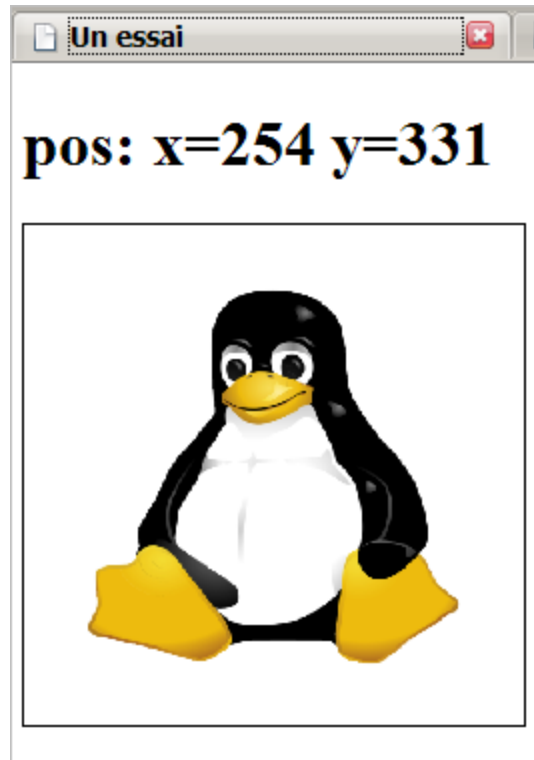
“ On peut ajouter des gestionnaires d'événement à tous les éléments.
Tous les gestionnaires ne s'appliquent pas à élément (Ex `onsubmit`
pour `<form>`). ”

Attribuer l'événement `click` à `<p>` :

```
<p onclick="ma_fonction()">...</p>
```


8.8.2. Exemple d'utilisation d'événement JS: **onmousemove**

```
<!DOCTYPE html >
<head>
  <script type="text/javascript">
function ca_bouge(event) {
  var el=document.getElementById("posh1");
  el.textContent="pos: x="+event.clientX+
    " y="+event.clientY;
}
</script>
</head>
<body>
  <h1 id="posh1">Texte</h1>
  <p> </p>
</body>
</html>
```

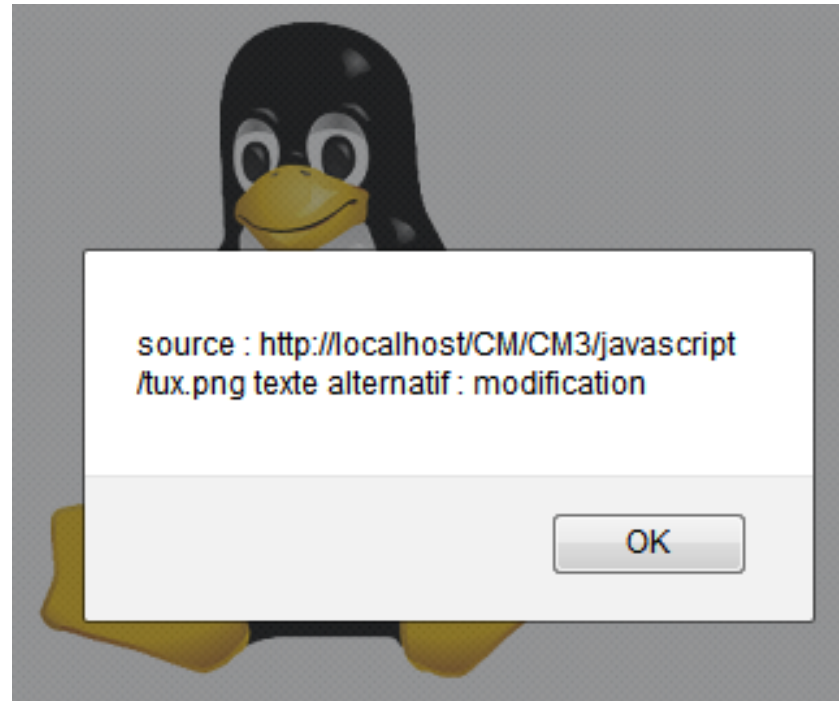


8.8.3. Mot clé **this**

“ Le mot-clé `this` est très utile puisqu'il permet de passer en paramètre d'une fonction javascript l'élément HTML sur lequel a porté l'événement. ”

```

<script>
function affiche(image){
alert("source : " + image.src + " texte alternatif : " + image.alt);
}
</script>
```



“ Affichage source et texte alternatif de l'image passée en paramètre. ”

8.8.4. Attribuer dynamiquement des événements en utilisant le DOM HTML

DOM HTML permet d'affecter des événements aux éléments HTML en utilisant JavaScript:

```
<script>
document.getElementById("demo").onclick=function(){displayDate()};
document.getElementById("demo").onclick=null; // pour dévalider
</script>
```

“ On peut par programmation décider d'affecter des événements à des objets HTML ”

8.9. JS peut modifier le style CSS des éléments **HTML**

```
document.getElementById(id).style.property="new_style"
```

Les attribus **CSS** dont le nom comporte un - doivent être modifiés :
`background-color` devient `backgroundColor` ...

Il est préférable de modifier la classe de l'élément avec la propriété `className` .

Rappel : Les information de mise en page doivent toutes être dans les fichiers de style (`CSS`), et non pas dans le code javascript

On crée dans le fichier `CSS` une classe `"selectionne"` :

```
.selectionne {          background-color: red;          }
```

et dans le code javascript, on applique cette classe à l'élément:

```
var laLigne = document.getElementById("ligne")  
laLigne.className="selectionne";
```

8.10. Javascript et Formulaire

```
<body>
  <form id="frm1" action="page.php" method="post" onsubmit="return valider();">
    <input type="text" id="txt1" ... />
  </form>
  ...
```

- `valider()` retourne **vrai**, le formulaire est soumis et la page spécifiée dans la propriété `action` de la balise `<form>` est appelée.
- `valider()` retourne **faux**, le formulaire n'est pas soumis, Javascript bloque ainsi l'envoi du formulaire.

“ La fonction `valider` doit donc renvoyer **faux** tant qu'il subsiste des erreurs de saisie dans le formulaire (champs mal remplis, etc.) ”

`forms` : donne un tableau des formulaires d'une page

Remarques : `images` : tableau des images, `links` : tableau des liens.

8.11. Javascript et la fenêtre du navigateur

`Window` représente la fenêtre du navigateur et permet d'accéder au document `HTML`, à l'historique, au navigateur utilisé ou à la barre d'état.

Variables globales

```
<script ...>mavariabile="toto";</script>
```

```
window.mavariabile
```

Propriétés et méthodes:

- `location` : @ page courante `window.location="http://www.page.fr"`
- `open()` : ouvre une nouvelle fenetre
`window.open("http://www.page.fr/");`
- `setTimeout()` : appeler une fonction dans x millisecondes
`window.setTimeout(bouger, 100);`

9. La méthode fetch en JavaScript

9.1. Introduction à fetch

La méthode fetch est une API moderne en JavaScript qui permet de faire des requêtes HTTP asynchrones vers un serveur avec `async/await`

Elle est basée sur les promesses (Promises) et remplace progressivement l'ancienne méthode AJAX avec XMLHttpRequest.

9.2. Récupérer des données (GET)

```
<!DOCTYPE html>
<html lang="fr">

<head>
  <meta charset="utf-8" />
  <script type="text/javascript" src="ajaxfetch.js"></script>
  <title> Essai Fetch </title>
</head>

<body>
  <h1> Essai Fetch Interne </h1>
  <button onclick="lireVins()">Requête GET</button>
  <div id="lecture"></div>
</body>

</html>
```

```

async function lireVins() {
  try {/* appel interne du script getCave.php */
    const reponse = await fetch('getCave.php'); // Attend la réponse du serveur.
    if (!reponse.ok) throw new Error(`Erreur : ${reponse.status}`);
    // Vérifie si la requête a réussi.

    console.log(reponse)

    const cave = await reponse.text(); //Convertit la réponse en texte (ici tableau HTML).
    console.log(cave)
    console.log("Status Code:", reponse.status, "Status Text:", reponse.statusText);

    document.getElementById("cave").innerHTML = cave;//écrit dans la page HTML

  } catch (error) { //Gère les erreurs.
    console.error('Erreur :', error);
  }
}

```

```
<?php
    $maBDD = new PDO('sqlite:bdd/cave.sqlite');
    $maBDD->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql="SELECT * FROM vins";
    $result = $maBDD->query($sql); // on exécute la requête
    if ($result)
    {
        $tableau = $result->fetchAll(PDO::FETCH_ASSOC); //parcours des résultats
        afficheTableau($tableau); // Retourne du code HTML mis en forme
    }
    else {
        echo '<p>'. 'Requête ne retournant pas de résultat ou BDD introuvable' . '</p>';
    }
?>
```

Essai Fetch Interne

Requête GET

Selection

noV	CRU	COULEUR	ORIGINE
V1	Beaujolais	Rouge	Bourgogne
V10	Sancerre	Rosé	Loire

[🔍](#) Inspecteur [📄](#) Console [🔧](#) Débogueur [↕](#) Réseau [🛠](#) Éditeur de style [🕒](#) Performances [🧠](#) Mémoire [📁](#) Stockage [🧑](#) Accessibilité [📱](#) Applications 1 [🔗](#) [⋮](#) [✕](#)

[🗑](#) Filtre les URL

[||](#) [+](#) [🔍](#) [🚫](#) [Tout](#) HTML CSS JS XHR Polices Images Médias WS Autre ☒ Désactiver le cache Aucune limitation de la bande passante ⚙

État	Méth...	Domaine	Fichier	Initiateur	Type	Transfert	Taille	En-têtes	Cookies	Requête	Réponse	Délais	Trace de la pile	
200	GET	127.0.0.1	Ajax_cm1_fetch.php	document	html	925 o	670 o	HTML						Brut ☐
200	GET	127.0.0.1	ajaxfetch.js	script	js	2,39 Ko	2,08 Ko							
404	GET	127.0.0.1	favicon.ico	FaviconLoader.s...	html	535 o	295 o							
200	GET	127.0.0.1	getCave.php	ajaxfetch.js:3 (fe...	html	1,16 Ko	910 o							

[🕒](#) 4 requêtes

3,95 Ko / 5,02 Ko transférés

Terminé en : 3,27 s

DOMContentLoaded: 29 ms

load: 31 ms

noV	CRU	COULEUR	ORIGINE
V1	Beaujolais	Rouge	Bourgogne
V10	Sancerre	Rosé	Loire
V2	Chablis	Blanc	Bourgogne
V3	Médoc	Rouge	Bordeaux
V4	St-Emilion	Rouge	Bordeaux
V5	Sauterne	Blanc	Bordeaux
V6	Chateaufort du Pape	Rouge	Côtes du Rhône
V7	Tavel	Rosé	Côtes du Rhône
V8	Gewurztraminer	Blanc	Alsace
V9	Sancerre	Blanc	Loire

9.3. Envoyer des données (POST)

```
async function creerVin() {
  try {
    const reponse = await fetch('postCave.php', {
      method: 'POST', // Type de requête
      headers: { "Content-Type": "application/x-www-form-urlencoded" }, //Format des données
      body: "cle=V12&cru=Médoc&couleur=Rouge&origine=Bordeaux" //Données à envoyer
    });
    if (!reponse.ok) throw new Error(`Erreur : ${reponse.status}`);
    const nouveauVin = await reponse.text();
    console.log('Vin créé :', nouveauVin);
    document.getElementById("creation").innerHTML = nouveauVin;
  } catch (error) {
    console.error('Erreur :', error);
  }
}
```

V7	Tavel	Rosé	Côtes du Rhône
V8	Gewurztraminer	Blanc	Alsace
V9	Sancerre	Blanc	Loire
V15	Modification	Languedoc	Rouge

Requête POST

1

Requête DELETE

Essai Fetch Externe

Requête GET Fetch Externe



Inspecteur



Console



Débogueur



Réseau



Éditeur de style



Performances



Mémoire



Stockage



Accessibilité



Applications



Filtrer les URL

||

+

🔍

🚫

Tout

HTML

CSS

JS

XHR

Polices

Images

Médias

WS

Autre

☒ Désactiver le cache

État	Méth...	Domaine	Fichier	Initiateur	Type	Transfert	Taille	 En-têtes Cookies Requête Réponse
200	GET	🔒 127.0.0.1	Ajax_cm1_fetch.php	document	html	925 o	670 o	
200	GET	🔒 127.0.0.1	ajaxfetch.js	script	js	2,39 Ko	2,08 Ko	Données de formulaire
404	GET	🔒 127.0.0.1	favicon.ico	FaviconLoader.s...	html	535 o	295 o	cle: "V12"
200	GET	🔒 127.0.0.1	getCave.php	ajaxfetch.js:3 (fe...	html	1,16 Ko	910 o	cru: "Médoc"
200	POST	🔒 127.0.0.1	postCave.php	ajaxfetch.js:23 (f...	html	254 o	1 o	couleur: "Rouge"
								origine: "Bordeaux"

9.4. Gestion des erreurs

- Toujours utiliser `try/catch` pour attraper les erreurs.
- Vérifier `reponse.ok` pour détecter les erreurs HTTP (ex : 404, 500).



IUT Lannion © 2026 by R&T is licensed under CC BY-NC-ND 4.0

Ce document est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 4.0 International